

Fəsil 2. End-to-End Machine Learning Layihəsi

Bu fəsildə siz daşınmaz əmlak şirkətində yeni işə götürülmüş bir data scientist rolunu oynayaraq nümunəvi bir layihəni başdan-sona (end to end) işləyəcəksiniz. Bu nümunə uydurmadır; məqsəd daşınmaz əmlak biznesi haqqında nəşə öyrətmək deyil, bir machine learning layihəsinin əsas addımlarını göstərməkdir. Birlikdə keçəcəyimiz əsas addımlar bunlardır:

1. Ümumi mənzərəyə bax (Look at the big picture).
2. Datanı əldə et (Get the data).
3. İçgörü (insight) qazanmaq üçün datanı araşdır və vizuallaşdır.
4. Datanı machine learning alqoritmləri üçün hazırla.
5. Bir model seç və onu öyrət (train).
6. Modelini fine-tune et.
7. Həllini təqdim et.
8. Sistemini işə sal (launch), nəzarət et (monitor) və saxlanılmasını təmin et (maintain).

Real Data ilə İşləmək

Machine learning öyrənərkən süni dataset-lər yerinə real dünya datası ilə təcrübə aparmaq ən yaxşıdır. Xoşbəxtlikdən, müxtəlif sahələri əhatə edən minlərlə açıq (open) dataset arasından seçim etmək mümkündür. Data əldə etmək üçün baxa biləcəyiniz bir neçə yer:

Populyar açıq data repozitoriyaları:

[OpenML.org](https://openml.org)

[Kaggle.com](https://kaggle.com)

[PapersWithCode.com](https://paperswithcode.com)

[UC Irvine Machine Learning Repository](https://machinelearningrepository.ucirvine.edu/)

[Amazon-un AWS dataset-ləri](https://aws.amazon.com/datasets/)

[TensorFlow dataset-ləri](https://tfhub.dev/)

Meta portallar (açıq data repozitoriyalarının siyahısını verirlər):

[DataPortals.org](https://dataportals.org)

OpenDataMonitor.eu

Çoxlu populyar açıq data repozitoriyasını sadalayan digər səhifələr:

[Wikipedia-nın machine learning dataset-ləri siyahısı](https://en.wikipedia.org/wiki/List_of_machine_learning_datasets)

Bu fəsildə biz StatLib repozitoriyasından¹ California Housing Prices dataset-indən istifadə edəcəyik (Şəkil 2-1-ə bax). Bu dataset 1990-cı il California siyahıyaalmasının (census) datasına əsaslanır. O, tam olaraq yeni deyil (o vaxt Bay Area-da gözəl bir ev hələ də əlçatan idi), lakin öyrənmək üçün çoxlu üstünlüyə malikdir, ona görə də onu yeni data kimi qəbul edəcəyik. Tədris məqsədilə mən bir categorical attribute əlavə etdim və bir neçə feature-u sildim.

Şəkil 2-1. California ev qiymətləri

Ümumi Mənzərəyə Bax

Machine Learning Housing Corporation-a xoş gəldiniz! İlk vəzifəniz California siyahıyaalma (census) datasından istifadə edərək ştatdakı ev qiymətlərinin modelini qurmaqdır. Bu data California-da hər block group üçün əhali (population), median income və median ev qiyməti kimi metrikləri ehtiva edir. Block group-lar ABŞ Census Bureau-nun nümunə (sample) datası dərc etdiyi ən kiçik coğrafi vahiddir (block group adətən 600-dən 3000-ə qədər əhaliyə malik olur). Mən onları qısa şəkildə “rayonlar” (districts) adlandıracağam.

Modeliniz bu datadan öyrənməli və bütün digər metriklər verildikdə istənilən rayonda median ev qiymətini proqnozlaşdırma (predict) bilməlidir.

MƏSLƏHƏT

Siz nizamlı bir data scientist olduğunuza görə, ilk növbədə machine learning layihə checklist-inizi çıxarmalısınız. Appendix A-dakı checklist ilə başlaya bilərsiniz; o, əksər machine learning layihələri üçün kifayət qədər yaxşı işləyir, lakin onu öz ehtiyaclarınıza uyğunlaşdırmağınızdan əmin olun. Bu fəsildə biz bir çox checklist maddəsini keçəcəyik, lakin bəzilərini də ötürəcəyik — ya özlüyündə aydın olduğuna, ya da sonrakı fəsillərdə müzakirə olunacağına görə.

Problemi Çərçivələ (Frame the Problem)

Müdirinizə verəcəyiniz ilk sual biznes məqsədinin (business objective) tam olaraq nə olduğudur. Model qurmaq, ehtimal ki, son məqsəd deyil. Şirkət bu modeldən necə istifadə etməyi və ondan necə fayda görməyi gözləyir? Məqsədi bilmək vacibdir, çünki o, problemi necə çərçivələdiyinizi, hansı alqoritmləri seçəcəyinizi, modelinizi qiymətləndirmək üçün hansı performance measure-dən istifadə edəcəyinizi və onu tənzimləməyə (tweaking) nə qədər səy sərf edəcəyinizi müəyyən edəcək.

Müdiriniz cavab verir ki, modelinizin çıxışı (bir rayonun median ev qiymətinin proqnozu) bir çox digər siqnalla (signal) birlikdə başqa bir machine learning sisteminə (Şəkil 2-2-yə

bax) ötürüləcək.² Bu downstream sistem müəyyən bir ərazidə investisiya qoymağın dəyərli olub-olmadığını müəyyən edəcək. Bunu düzgün etmək kritikdir, çünki o, birbaşa gəlirə (revenue) təsir edir.

Müdirinizə verəcəyiniz növbəti sual cari həllin (varsa) necə göründüyüdür. Cari vəziyyət çox vaxt sizə həm performans üçün istinad nöqtəsi, həm də problemi necə həll etmək barədə içgörü verir. Müdiriniz cavab verir ki, rayon ev qiymətləri hazırda ekspertlər tərəfindən əl ilə təxmin edilir: bir komanda rayon haqqında ən son məlumatı toplayır və median ev qiymətini əldə edə bilmədikdə, onu mürəkkəb qaydalarla təxmin edirlər.

Şəkil 2-2. Daşınmaz əmlak investisiyaları üçün machine learning pipeline

Bu, baha başa gəlir və vaxt aparır, üstəlik onların təxminləri də o qədər yaxşı deyil; faktiki median ev qiymətini öyrənə bildikləri hallarda çox vaxt başa düşürlər ki, təxminləri 30%-dən çox yanlış olub. Buna görə də şirkət düşünür ki, bir rayon haqqında digər data verildikdə həmin rayonun median ev qiymətini proqnozlaşdırmaq üçün bir modeli öyrətmək faydalı olardı. Siyahıyaalma datası bu məqsəd üçün istifadə etmək üçün əla dataset kimi görünür, çünki o, minlərlə rayonun median ev qiymətlərini və digər datanı ehtiva edir.

PIPELINE-LAR

Data emal (processing) komponentlərinin ardıcılığına data *pipeline* deyilir. Pipeline-lar machine learning sistemlərində çox geniş yayılmışdır, çünki manipulyasiya ediləcək çoxlu data və tətbiq ediləcək çoxlu data transformasiyası olur.

Komponentlər adətən asinxron (asynchronously) işləyir. Hər komponent böyük həcmdə datanı içəri çəkir, onu emal edir və nəticəni başqa bir data store-a ötürür. Sonra, müəyyən vaxtdan sonra, pipeline-dakı növbəti komponent bu datanı içəri çəkir və öz çıxışını verir. Hər komponent kifayət qədər müstəqildir (self-contained): komponentlər arasındakı interfeys sadəcə data store-dur. Bu, sistemi anlamağı asanlaşdırır (data flow graph-ın köməyi ilə) və müxtəlif komandalar müxtəlif komponentlərə diqqət yetirə bilər. Üstəlik, bir komponent sındıqda, downstream komponentlər çox vaxt sadəcə sınımış komponentin son çıxışından istifadə edərək normal işləməyə davam edə bilər (heç olmasa bir müddət). Bu, arxitekturanı kifayət qədər dayanıqlı (robust) edir.

Digər tərəfdən, düzgün monitoring tətbiq edilməyibsə, sınımış bir komponent bir müddət fərq edilməyə bilər. Data köhnəlir (stale) və ümumi sistemin performansı düşür.

Bütün bu məlumatlarla siz indi sisteminizi dizayn etməyə hazırsınız. Əvvəlcə modelin hansı növ training supervision-a ehtiyacı olacağını müəyyən edin: bu supervised, unsupervised, semi-supervised, self-supervised, yoxsa reinforcement learning tapşırığıdır? Və bu classification tapşırığı, regression tapşırığı, yoxsa başqa bir şeydir? Batch learning, yoxsa online learning texnikalarından istifadə etməlisiniz? Davam etməzdən əvvəl dayanın və bu suallara özünüz cavab verməyə çalışın.

Cavabları tapdınız? Gəlin görək. Bu, açıq-aşkar tipik bir supervised learning tapşırığıdır, çünki model *labeled* nümunələrlə öyrədilə bilər (hər instance gözlənilən çıxışla, yəni rayonun median ev qiyməti ilə gəlir). Bu, tipik bir regression tapşırığıdır, çünki modeldən bir dəyəri proqnozlaşdırmaq tələb olunacaq. Daha dəqiq desək, bu bir *multiple regression* problemidir, çünki sistem proqnoz vermək üçün çoxlu feature-dan istifadə edəcək (rayonun population-u, median income və s.). Bu, həmçinin bir *univariate regression* problemidir, çünki biz hər rayon üçün yalnız bir dəyəri proqnozlaşdırmağa çalışırıq. Əgər hər rayon üçün çoxlu dəyəri proqnozlaşdırmağa çalışsaydıq, bu bir *multivariate regression* problemi olardı. Nəhayət, sistemə daxil olan davamlı data axını yoxdur, dəyişən dataya sürətlə uyğunlaşmaq üçün xüsusi bir ehtiyac yoxdur və data yaddaşa (memory) sığacaq qədər kiçikdir, ona görə də adi batch learning işi görəcək.

MƏSLƏHƏT

Əgər data nəhəng olsaydı, siz ya batch learning işinizi çoxlu server arasında bölə bilərdiniz (MapReduce texnikasından istifadə edərək), ya da online learning texnikasından istifadə edə bilərdiniz.

Performance Measure Seç

Növbəti addımınız bir performance measure seçməkdir. Regression problemləri üçün tipik bir performance measure *root mean square error* (RMSE)-dir. O, sistemin öz proqnozlarında adətən nə qədər səhv etdiyi barədə təsəvvür yaradır və böyük səhvlərə daha yüksək çəki verir. Tənlik 2-1 RMSE-ni hesablamaq üçün riyazi düsturu göstərir.

Tənlik 2-1. Root mean square error (RMSE)

$$RMSE(X, h) = \frac{1}{m} \sum_{i=1}^m h(x(i)) - y(i)^2$$

NOTASIYALAR

Bu tənlik bu kitab boyunca istifadə edəcəyim bir neçə çox geniş yayılmış machine learning notasiyasını təqdim edir:

- m RMSE-ni ölçdüyünüz dataset-dəki instance-ların sayıdır.

For example, əgər siz RMSE-ni 2000 rayonluq bir validation set üzərində qiymətləndirirsinizsə, onda $m = 2000$.

- $x(i)$ dataset-dəki i -ci instance-ın bütün feature dəyərlərinin (label istisna olmaqla) vektorudur və $y(i)$ onun label-ıdır (həmin instance üçün istənilən çıxış dəyəri).

For example, əgər dataset-dəki ilk rayon -118.29° uzunluq dairəsində (longitude), 33.91° en dairəsində (latitude) yerləşirsə və 1416 sakini ilə median income \$38,372 və median ev dəyəri \$156,400-dırsa (hələlik digər feature-ları nəzərə almasaq), onda:

$$x(1) = -118.29 \ 33.91 \ 1,416 \ 38,372$$

və:

$$y(1) = 156,400$$

X dataset-dəki bütün instance-ların bütün feature dəyərlərini (label-lar istisna olmaqla) ehtiva edən matrisdir. Hər instance üçün bir sətir var və i -ci sətir $\mathbf{x}(i)$ -nin transpozisiyasına (transpose) bərabərdir, $(\mathbf{x}(i))^T$ kimi qeyd olunur.³

For example, əgər ilk rayon təsvir olunduğu kimidirsə, onda **X** matrisi belə görünür:

$$X = (\mathbf{x}(1))^T (\mathbf{x}(2))^T \dots (\mathbf{x}(1999))^T (\mathbf{x}(2000))^T = -118.29 \ 33.91 \ 1,416 \ 38,372 \dots$$

h sizin sisteminizin proqnoz funksiyasıdır, həm də *hypothesis* adlanır. Sisteminizə bir instance-ın feature vektoru $\mathbf{x}(i)$ verildikdə, o, həmin instance üçün proqnozlaşdırılmış $\hat{y}(i) = h(\mathbf{x}(i))$ dəyərini verir ($\hat{y}$ “y-hat” kimi tələffüz olunur).

For example, əgər sisteminiz ilk rayonda median ev qiymətinin \$158,400 olduğunu proqnozlaşdırırsa, onda $\hat{y}(1) = h(\mathbf{x}(1)) = 158,400$. Bu rayon üçün proqnoz səhvi $\hat{y}(1) - y(1) = 2,000$ -dir.

$RMSE(\mathbf{X}, h)$ sizin h hypothesis-inizdən istifadə edərək nümunələr toplusu üzərində ölçülən cost function-dır.

Biz skalyar dəyərlər (məsələn, $y(i)$ və ya m) və funksiya adları (məsələn, h) üçün kiçik kursiv (italic) şrift, vektorlar (məsələn, $\mathbf{x}(i)$) üçün kiçik qalın (bold) şrift, və matrislər (məsələn, **X**) üçün böyük qalın şrift istifadə edirik.

RMSE ümumiyyətlə regression tapşırıqları üçün üstünlük verilən performance measure olsa da, bəzi kontekstlərdə başqa bir funksiyaadan istifadə etməyə üstünlük verə bilərsiniz. For example, çoxlu outlier rayon varsa. Bu halda, Tənlik 2-2-də göstərilən *mean absolute error* (MAE, həmçinin *average absolute deviation* adlanır) istifadə etməyi düşünə bilərsiniz:

Tənlik 2-2. Mean absolute error (MAE)

$$MAE(\mathbf{X}, h) = \frac{1}{m} \sum_{i=1}^m |h(\mathbf{x}(i)) - y(i)|$$

Həm RMSE, həm də MAE iki vektor arasındakı məsafəni ölçmək üsullarıdır: proqnozlar vektoru və hədəf (target) dəyərlər vektoru. Müxtəlif məsafə ölçüləri, və ya *norm*-lar mümkündür:

- Kvadratlar cəminin kökünü hesablamaq (RMSE) *Euclidean norm*-a uyğundur: bu, hamımıza tanış olan məsafə anlayışıdır. O, həmçinin ℓ_2 *norm* adlanır, $\|\cdot\|_2$ (və ya sadəcə $\|\cdot\|$) kimi qeyd olunur.
- Mütləq qiymətlər cəmini hesablamaq (MAE) ℓ_1 *norm*-a uyğundur, $\|\cdot\|_1$ kimi qeyd olunur. Buna bəzən *Manhattan norm* deyilir, çünki o, şəhərdə yalnız ortoqonal şəhər

bloklarıyla hərəkət edə bildiyiniz halda iki nöqtə arasındakı məsafəni ölçür.

- Daha ümumi olaraq, n element ehtiva edən $\mathbf{v}$ vektorunun ℓ_k norm-u belə təyin olunur:
$$\|\mathbf{v}\|_k = (|v_1|^k + |v_2|^k + \dots + |v_n|^k)^{1/k}.$$
 ℓ_0 vektordakı sıfırdan fərqli elementlərin sayını verir, ℓ_∞ isə vektordakı maksimum mütləq qiyməti verir.

Norm indeksi nə qədər yüksəkdirsə, o, böyük dəyərlərə bir o qədər çox diqqət yetirir və kiçik dəyərləri laqeyd edir. Buna görə də RMSE outlier-lərə MAE-dən daha həssasdır. Lakin outlier-lər eksponensial olaraq nadirdirsə (zəng formalı (bell-shaped) əyridə olduğu kimi), RMSE çox yaxşı işləyir və ümumiyyətlə üstünlük verilir.

Fərziyyələri Yoxla (Check the Assumptions)

Nəhayət, indiyə qədər (sizin və ya başqaları tərəfindən) edilmiş fərziyyələri (assumptions) siyahıya almaq və yoxlamaq yaxşı təcrübədir; bu, ciddi problemləri erkən mərhələdə tutmağa kömək edə bilər. For example, sisteminizin verdiyi rayon qiymətləri downstream machine learning sisteminə ötürüləcək və siz fərz edirsiniz ki, bu qiymətlər elə bu cür istifadə olunacaq. Bəs downstream sistem qiymətləri kateqoriyalara (məsələn, “ucuz”, “orta” və ya “bahalı”) çevirib, sonra qiymətlərin özü əvəzinə həmin kateqoriyalardan istifadə edərsə nə olar? Bu halda, qiyməti mükəmməl dəqiqliklə tapmaq ümumiyyətlə vacib deyil; sisteminizə sadəcə kateqoriyanı düzgün tapmaq lazımdır. Əgər belədirsə, onda problem regression tapşırığı kimi deyil, classification tapşırığı kimi çərçivələnməli idi. Bunu bir regression sistemi üzərində aylarla işlədikdən sonra öyrənmək istəməzsiz.

Xoşbəxtlikdən, downstream sistemə cavabdeh komanda ilə danışdıqdan sonra əminsiz ki, onların həqiqətən faktiki qiymətlərə ehtiyacı var, təkcə kateqoriyalara deyil. Əla! Hər şey hazırdır, işıqlar yaşıldır və indi kod yazmağa başlaya bilərsiniz!

Datayı Əldə Et (Get the Data)

Əllərinizi kirlətmək vaxtıdır. Laptopunuzu götürüb kod nümunələrini addım-addım keçməkdən çəkinməyin. Müqəddimədə (preface) qeyd etdiyim kimi, bu kitabdakı bütün kod nümunələri açıq mənbə (open source) olub onlayn olaraq Jupyter notebook-ları kimi mövcuddur — bunlar mətn, şəkillər və icra edilə bilən kod parçalarını (bizim halda Python) ehtiva edən interaktiv sənədlərdir. Bu kitabda mən fərz edəcəyəm ki, siz bu notebook-ları Google Colab-da işlədirsiniz — bu, istənilən Jupyter notebook-u birbaşa onlayn işlətməyə imkan verən pulsuz xidmətdir və maşınıza heç nə quraşdırmağa ehtiyac qalmır. Əgər başqa bir onlayn platformadan (məsələn, Kaggle) istifadə etmək istəyirsinizsə və ya hər şeyi öz maşınıızda lokal quraşdırmaq istəyirsinizsə, lütfən kitabın GitHub səhifəsindəki təlimatlara baxın.

Google Colab İstifadə Edərək Kod Nümunələrini İşlətmək

Əvvəlcə veb brauzer açın və <https://homl.info/colab3> ünvanına daxil olun: bu sizi Google Colab-a aparacaq və bu kitab üçün Jupyter notebook-larının siyahısını göstərəcək (Şəkil 2-

3-ə bax). Hər fəsil üçün bir notebook, üstəlik NumPy, Matplotlib, Pandas, xətti cəbr (linear algebra) və diferensial hesablama (differential calculus) üçün bir neçə əlavə notebook və təlimat tapacaqsınız. For example, *02_end_to_end_machine_learning_project.ipynb* üzərinə klik etsəniz, Fəsil 2-dən olan notebook Google Colab-da açılacaq (Şəkil 2-4-ə bax).

Jupyter notebook hüceyrələrin (cells) siyahısından ibarətdir. Hər hüceyrə ya icra edilə bilən kod, ya da mətn ehtiva edir. İlk mətn hüceyrəsinə (içində “Welcome to Machine Learning Housing Corp.!” cümləsi olan) iki dəfə klik etməyi sınayın. Bu, hüceyrəni redaktə üçün açacaq. Diqqət edin ki, Jupyter notebook-ları formatlaşdırma üçün Markdown sintaksisindən istifadə edir (məsələn, **qalın**, *kursiv*, # Başlıq, [url](link mətni) və s.). Bu mətni dəyişdirməyi sınayın, sonra nəticəni görmək üçün Shift-Enter basın.

Şəkil 2-3. Google Colab-dakı notebook-ların siyahısı

Şəkil 2-4. Google Colab-dakı notebook-unuz

Sonra, menyudan Insert → “Code cell” seçərək yeni bir kod hüceyrəsi yaradın. Alternativ olaraq, alətlər panelindəki + Code düyməsinə klik edə bilərsiniz, və ya siçanı bir hüceyrənin altına gətirin və + Code və + Text görünənə qədər gözləyin, sonra + Code-a klik edin. Yeni kod hüceyrəsində bir az Python kodu yazın, məsələn `print("Hello World")`, sonra bu kodu işlətmək üçün Shift-Enter basın (və ya hüceyrənin sol tərəfindəki ▷ düyməsinə klik edin).

Əgər Google hesabınıza daxil olmamısınızsa, indi daxil olmağınız istəniləcək (əgər hələ Google hesabınız yoxdursa, bir hesab yaratmalı olacaqsınız). Daxil olduqdan sonra kodu işlətməyə cəhd etdiyiniz zaman bu notebook-un Google tərəfindən yazılmadığını bildirən bir təhlükəsizlik xəbərdarlığı görəcəksiniz. Zərərli bir şəxs sizi Google etimadnamələrinizi (credentials) daxil etməyə aldatmağa çalışan bir notebook yarada bilər ki, şəxsi datanıza giriş əldə etsin, ona görə də bir notebook işlətməzdən əvvəl həmişə onun müəllifinə etibar etdiyinizdən əmin olun (və ya işlətməzdən əvvəl hər kod hüceyrəsinin nə edəcəyini iki dəfə yoxlayın). Mənə etibar etdiyinizi (və ya hər kod hüceyrəsini yoxlamağı planladığınızı) fərz etsək, indi “Run anyway”-ə klik edə bilərsiniz.

Sonra Colab sizin üçün yeni bir *runtime* ayıracaq: bu, Google serverlərində yerləşən, bir dəstə alət və Python kitabxanası (library) ehtiva edən pulsuz bir virtual maşındır və əksər fəsillər üçün lazım olan hər şeyi əhatə edir (bəzi fəsillərdə əlavə kitabxanalar quraşdırmaq üçün bir əmr işlətməli olacaqsınız). Bu, bir neçə saniyə çəkəcək. Sonra Colab avtomatik olaraq bu runtime-a qoşulacaq və yeni kod hüceyrənizi icra etmək üçün ondan istifadə edəcək. Vacibdir ki, kod runtime-da işləyir, *sizin maşınıınızda deyil*. Kodun çıxışı hüceyrənin altında göstəriləcək. Təbriklər, Colab-da bir az Python kodu işlətdiniz!

MƏSLƏHƏT

Yeni bir kod hüceyrəsi əlavə etmək üçün Ctrl-M (və ya macOS-da Cmd-M), ardınca A (aktiv hüceyrənin üstünə əlavə etmək üçün) və ya B (altına əlavə etmək üçün) də yaza bilərsiniz. Bir çox digər klaviatura qısayolları (keyboard shortcuts) mövcuddur: onları Ctrl-M (və ya

Cmd-M), sonra H yazaraq görə və redaktə edə bilərsiniz. Əgər notebook-ları Kaggle-də və ya öz maşınıızda JupyterLab və ya Jupyter eklentisi olan Visual Studio Code kimi bir IDE istifadə edərək işlətməyi seçsəniz, bəzi kiçik fərqlər görəcəksiniz — runtime-lar *kernel* adlanır, istifadəçi interfeysi və klaviatura qısayolları bir az fərqlidir və s. — lakin bir Jupyter mühitindən digərinə keçmək o qədər də çətin deyil.

Kod Dəyişikliklərinizi və Datanızı Saxlamaq

Colab notebook-una dəyişikliklər edə bilərsiniz və onlar brauzer tabınızı açıq saxladığınız müddətcə qalacaq. Lakin onu bağladıqdan sonra dəyişikliklər itəcək. Bunun qarşısını almaq üçün File → “Save a copy in Drive” seçərək notebook-un bir nüsxəsini Google Drive-ınıza saxladığınızdan əmin olun. Alternativ olaraq, File → Download → “Download .ipynb” seçərək notebook-u kompüterinizə yükləyə bilərsiniz. Sonra daha sonra <https://colab.research.google.com> ünvanına daxil olub notebook-u yenidən açə bilərsiniz (ya Google Drive-dan, ya da kompüterinizdən yükləyərək).

XƏBƏRDARLIQ

Google Colab yalnız interaktiv istifadə üçün nəzərdə tutulub: notebook-larda təcrübə apara və kodu istədiyiniz kimi dəyişdirə bilərsiniz, lakin notebook-ları uzun müddət nəzarətsiz işləməyə buraxa bilməzsəniz, əks halda runtime bağlanacaq və onun bütün datası itəcək.

Əgər notebook əhəmiyyət verdiyiniz data yaradırsa, runtime bağlanmazdan əvvəl bu datanı yüklədiyinizdən əmin olun. Bunun üçün Files ikonuna klik edin (Şəkil 2-5-də addım 1-ə bax), yükləmək istədiyiniz faylı tapın, onun yanındakı şaquli nöqtələrə klik edin (addım 2) və Download-a klik edin (addım 3). Alternativ olaraq, Google Drive-ınızı runtime-a mount edə bilərsiniz ki, notebook faylları sanki lokal qovluq imiş kimi birbaşa Google Drive-a oxuyub yaza bilsin. Bunun üçün Files ikonuna (addım 1), sonra Google Drive ikonuna klik edin (Şəkil 2-5-də dairəyə alınıb) və ekrandakı təlimatları izləyin.

==> picture [79 x 54] intentionally omitted <==

----- Start of picture text -----
 '—@
----- End of picture text -----

==> picture [44 x 54] intentionally omitted <==

----- Start of picture text -----
 Refresh
----- End of picture text -----

Şəkil 2-5. Google Colab runtime-dan bir faylın yüklənməsi (addım 1-dən 3-ə qədər), və ya Google Drive-ınızın mount edilməsi (dairəyə alınmış ikon)

Default olaraq, Google Drive-ınız `/content/drive/MyDrive` ünvanında mount olunacaq. Bir data faylını yedəkləmək (back up) istəyirsinizsə, sadəcə onu bu qovluğa kopyalayın: `!cp /content/my_great_model /content/drive/MyDrive` işlədərək. Nida işarəsi (!) ilə başlayan istənilən əmr Python kodu kimi deyil, shell əmri kimi qəbul edilir: `cp` bir faylı bir yoldan

başqasına kopyalamaq üçün Linux shell əmridir. Qeyd edin ki, Colab runtime-ları Linux-da (xüsusilə Ubuntu) işləyir.

İnteraktivliyin Gücü və Təhlükəsi

Jupyter notebook-ları interaktivdir və bu əla şeydir: hər hüceyrəni bir-bir işlədə, istənilən nöqtədə dayana, bir hüceyrə əlavə edə, kodla təcrübə apara, geri qayıdıb eyni hüceyrəni yenidən işlədə bilərsiniz və s. — və mən bunu etməyinizi şiddətlə tövsiyə edirəm. Əgər hüceyrələri heç vaxt onlarla təcrübə etmədən sadəcə bir-bir işlədirsinizsə, o qədər sürətli öyrənməyəcəksiniz. Lakin bu çevikliyin bir bedeli var: hüceyrələri səhv ardıcılıqla işlətmək və ya bir hüceyrəni işlətməyi unutmaq çox asandır. Bu baş verərsə, sonrakı kod hüceyrələri çox güman ki, uğursuz olacaq. For example, hər notebook-dakı ilk kod hüceyrəsi quraşdırma (setup) kodunu (məsələn, import-lar) ehtiva edir, ona görə də onu birinci işlətdiyinizdən əmin olun, əks halda heç nə işləməyəcək.

MƏSLƏHƏT

Əgər nə vaxtsa qəribə bir səhvlə qarşılaşsanız, runtime-ı yenidən başlatmağı (menyudan Runtime → “Restart runtime” seçərək) və sonra notebook-un əvvəlindən bütün hüceyrələri yenidən işlətməyi sınayın. Bu, çox vaxt problemi həll edir. Əgər həll etmirsə, çox güman ki, etdiyiniz dəyişikliklərdən biri notebook-u sındırıb: sadəcə orijinal notebook-a qayıdın və yenidən sınayın. Əgər hələ də uğursuz olursa, lütfən GitHub-da bir issue açın.

Kitab Kodu və Notebook Kodu

Bəzən bu kitabdakı kod ilə notebook-lardakı kod arasında kiçik fərqlər görə bilərsiniz. Bu, bir neçə səbəbdən baş verə bilər:

- Bu sətirləri oxuyana qədər bir kitabxana bir az dəyişmiş ola bilər, ya da bütün söylərimə baxmayaraq mən kitabda bir səhv etmiş ola bilərəm. Təəssüf ki, mən sizin kitab nüsxənizdəki kodu seyrli şəkildə düzəldə bilmərəm (əgər siz elektron nüsxə oxumursunuz və ən son versiyayı yükləyə bilmirsinizsə), lakin notebook-ları *düzəldə bilərəm*. Ona görə də bu kitabdan kodu kopyaladıqdan sonra bir səhvlə qarşılaşsanız, lütfən notebook-larda düzəldilmiş kodu axtarın: mən onları səhvsiz və ən son kitabxana versiyaları ilə yenilənmiş saxlamağa çalışacağam.
- Notebook-lar şəkilləri gözəlləşdirmək (labellar əlavə etmək, şrift ölçülərini təyin etmək və s.) və onları bu kitab üçün yüksək çözünlükdə (resolution) saxlamaq üçün bir az əlavə kod ehtiva edir. İstəyirsinizsə, bu əlavə kodu rahatlıqla nəzərə almaya bilərsiniz.

Mən kodu oxunaqlılıq və sadəlik üçün optimallaşdırdım: onu mümkün qədər xətti və düz (flat) etdim, çox az funksiya və ya class təyin etdim. Məqsəd, işlətdiyiniz kodun ümumiyyətlə tam qarşınızda olmasını təmin etməkdir — axtarmalı olduğunuz bir neçə

abstraksiya qatının içinə yuvalanmış (nested) deyil. Bu, həm də kodla təcrübə aparmağı asanlaşdırır. Sadəlik üçün məhdud error handling var və ən az yayılmış import-ların bəzilərini fayl başlığında yerləşdirmək əvəzinə (PEP 8 Python style guide-ında tövsiyə olunduğu kimi) lazım olduqları yerdə qoydum. Bununla belə, sizin production kodunuz çox da fərqli olmayacaq: sadəcə bir az daha modulyar, əlavə test-lər və error handling ilə.

OK! Colab ilə rahat olduqdan sonra datanı yükləməyə hazırsınız.

Datanı Yüklə (Download the Data)

Tipik mühitlərdə datanız relational database və ya başqa bir geniş yayılmış data store-da mövcud olardı və çoxlu cədvəl/sənəd/fayl arasında paylanmış olardı. Ona giriş əldə etmək üçün əvvəlcə etimadnamələrinizi və giriş icazələrinizi almalı⁴ və data sxemi (schema) ilə tanış olmalı olardınız. Bu layihədə isə hər şey çox daha sadədir: siz sadəcə bütün datanı ehtiva edən *housing.csv* adlı vergüllə ayrılmış dəyərlər (comma-separated values, CSV) faylını ehtiva edən tək bir sıxılmış (compressed) fayl, *housing.tgz* yükləyəcəksiniz.

Datanı əl ilə yükləyib açmaq əvəzinə, bunu sizin üçün edən bir funksiya yazmaq adətən üstünlük verilir. Bu, xüsusilə data müntəzəm olaraq dəyişirsə faydalıdır: ən son datanı əldə etmək üçün funksiyadan istifadə edən kiçik bir skript yaza bilərsiniz (və ya bunu müntəzəm intervallarla avtomatik etmək üçün cədvəlli (scheduled) bir tapşırıq qura bilərsiniz). Datanı əldə etmə prosesini avtomatlaşdırmaq, həmçinin dataset-i çoxlu məşində quraşdırmaq lazım olduqda da faydalıdır.

Datanı əldə etmək və yükləmək üçün funksiya budur:

```
from pathlib import Path import pandas as pd import tarfile import urllib.request
```

```
def load_housing_data(): tarball_path = Path("datasets/housing.tgz") if not  
tarball_path.is_file(): Path("datasets").mkdir(parents= True , exist_ok= True ) url =  
"https://github.com/ageron/data/raw/main/housing.tgz" urllib.request.urlretrieve(url,  
tarball_path) with tarfile.open(tarball_path) as housing_tarball:  
housing_tarball.extractall(path="datasets") return  
pd.read_csv(Path("datasets/housing/housing.csv"))
```

```
housing = load_housing_data()
```

`load_housing_data()` çağırıldıqda, o, *datasets/housing.tgz* faylını axtarır. Əgər onu tapmasa, cari qovluğun içində (Colab-da default olaraq */content*) *datasets* qovluğunu yaradır, *ageron/data* GitHub repozitoriyasından *housing.tgz* faylını yükləyir və onun məzmununu *datasets* qovluğuna çıxarır; bu, içində *housing.csv* faylı olan *datasets/housing* qovluğunu yaradır. Nəhayət, funksiya bu CSV faylını bütün datanı ehtiva edən Pandas DataFrame obyektinə yükləyir və onu qaytarır.

Data Strukturuna Tez Bir Nəzər Sal

DataFrame-in head() metodundan istifadə edərək datanın ilk beş sətirinə baxmaqla başlayırırsınız (Şəkil 2-6-ya bax).

Şəkil 2-6. Dataset-dəki ilk beş sətir

Hər sətir bir rayonu təmsil edir. 10 attribute var (hamısı ekran görüntüsündə göstərilməyib): longitude, latitude, housing_median_age, total_rooms, total_bedrooms, population, households, median_income, median_house_value və ocean_proximity.

info() metodu datanın tez təsvirini əldə etmək üçün faydalıdır, xüsusilə sətirlərin ümumi sayını, hər attribute-un tipini və null olmayan (non-null) dəyərlərin sayını:

```
>>> housing.info() <class 'pandas.core.frame.DataFrame'> RangeIndex: 20640 entries, 0 to 20639 Data columns (total 10 columns): # Column Non-Null Count Dtype
```

-
- 0 longitude 20640 non-null float64
 - 1 latitude 20640 non-null float64
 - 2 housing_median_age 20640 non-null float64
 - 3 total_rooms 20640 non-null float64
 - 4 total_bedrooms 20433 non-null float64
 - 5 population 20640 non-null float64
 - 6 households 20640 non-null float64
 - 7 median_income 20640 non-null float64
 - 8 median_house_value 20640 non-null float64
 - 9 ocean_proximity 20640 non-null object dtypes: float64(9), object(1)

memory usage: 1.6+ MB

QEYD

Bu kitabda, burada olduğu kimi bir kod nümunəsi kod və çıxışların qarışığını ehtiva etdikdə, daha yaxşı oxunaqlılıq üçün Python interpretatoru kimi formatlaşdırılır: kod sətirləri >>> (və ya girintili (indented) bloklar üçün ...) ilə önləşdirilir, çıxışların isə önlük (prefix) olmur.

Dataset-də 20,640 instance var ki, bu da machine learning standartlarına görə kifayət qədər kiçikdir, lakin başlamaq üçün mükəmməldir. Diqqət edirsiniz ki, total_bedrooms attribute-unun yalnız 20,433 null olmayan dəyəri var, yəni 207 rayonda bu feature çatışmır. Bununla sonra məşğul olmalı olacaqsınız.

ocean_proximity istisna olmaqla bütün attribute-lar ədədidir (numerical). Onun tipi object-dir, ona görə də istənilən növ Python obyektini saxlaya bilər. Lakin bu datanı CSV faylından yüklədiyiniz üçün bilirsiniz ki, o, mətn (text) attribute-u olmalıdır. İlk beş sətirə baxdığınız zaman, ehtimal ki, ocean_proximity sütunundakı dəyərlərin təkrarlanan olduğunu fərq etdiniz, yəni bu, ehtimal ki, categorical attribute-dur. value_counts() metodundan istifadə edərək hansı kateqoriyaların mövcud olduğunu və hər kateqoriyaya neçə rayonun aid olduğunu öyrənə bilərsiniz:

```
>>> housing["ocean_proximity"].value_counts() <1H OCEAN 9136 INLAND 6551 NEAR  
OCEAN 2658 NEAR BAY 2290 ISLAND 5 Name: ocean_proximity, dtype: int64
```

Gəlin digər sahələrə baxaq. describe() metodu numerical attribute-ların xülasəsini göstərir (Şəkil 2-7).

Şəkil 2-7. Hər numerical attribute-un xülasəsi

count, mean, min və max sətirləri özlüyündə aydındır. Qeyd edin ki, null dəyərlər nəzərə alınmır (məsələn, total_bedrooms-un count-u 20,640 deyil, 20,433-dür). std sətiri *standard deviation*-ı göstərir ki, bu da dəyərlərin nə qədər dağınıq olduğunu ölçür.⁵ 25%, 50% və 75% sətirləri müvafiq *persentil*-ləri (percentiles) göstərir: persentil müşahidələr qrupunda müşahidələrin müəyyən faizinin altında qaldığı dəyəri göstərir. For example, rayonların 25%-nin housing_median_age-i 18-dən aşağıdır, 50%-i 29-dan aşağı, 75%-i isə 37-dən aşağıdır. Bunlar çox vaxt 25-ci persentil (və ya birinci *kvartil* (quartile)), median və 75-ci persentil (və ya üçüncü kvartil) adlanır.

Məşğul olduğunuz datanın növü haqqında təsəvvür yaratmağın başqa bir tez yolu hər numerical attribute üçün histogram çəkməkdir. Histogram müəyyən dəyər aralığına (üfüqi oxda) malik instance-ların sayını (şaquli oxda) göstərir. Bunu ya bir attribute-u bir dəfəyə çəkmə bilərsiniz, ya da bütün dataset üzərində hist() metodunu çağırma bilərsiniz (aşağıdakı kod nümunəsində göstərildiyi kimi) və o, hər numerical attribute üçün bir histogram çəkəcək (Şəkil 2-8-ə bax):

```
import matplotlib.pyplot as plt
```

```
housing.hist(bins=50, figsize=(12, 8)) plt.show()
```

Şəkil 2-8. Hər numerical attribute üçün bir histogram

Bu histogramlara baxaraq bir neçə şeyi fərq edirsiniz:

- Əvvəlcə, median income attribute-u ABŞ dolları (USD) ilə ifadə olunmuş kimi görünür. Datanı toplayan komanda ilə yoxladıqdan sonra sizə deyilir ki, data miqyaslandırılıb (scaled) və daha yüksək median income-lar üçün 15-də (əslində 15.0001), daha aşağı median income-lar üçün isə 0.5-də (əslində 0.4999) kəsilib (capped). Rəqəmlər təxminən on minlərlə dolları təmsil edir (məsələn, 3 əslində təxminən \$30,000 deməkdir). Öncədən emal olunmuş (preprocessed) attribute-larla

işləmək machine learning-də adi haldır və mütləq problem deyil, lakin datanın necə hesablandığını anlamağa çalışmalısınız.

- housing median age və median ev dəyəri də kəsilib (capped). Sonuncu ciddi problem ola bilər, çünki o, sizin hədəf (target) attribute-unuzdur (sizin label-larınız). Machine learning alqoritmləriniz qiymətlərin heç vaxt həmin həddi keçmədiyini öyrənə bilər. Bunun problem olub-olmadığını görmək üçün müştəri komandanızla (sisteminizin çıxışından istifadə edəcək komanda) yoxlamalısınız. Əgər sizə deyirlərsə ki, hətta \$500,000-dən çox üçün də dəqiq proqnozlara ehtiyacları var, onda iki seçiminiz var:
 - Label-ları kəsilmiş rayonlar üçün düzgün label-lar toplayın.
 - Həmin rayonları training set-dən silin (həm də test set-dən, çünki sisteminiz \$500,000-dən çox dəyərlər proqnozlaşdırırsa, zəif qiymətləndirilməməlidir).
- Bu attribute-ların çox fərqli miqyasları (scales) var. Bunu bu fəsildə daha sonra, feature scaling-i araşdırarkən müzakirə edəcəyik.
- Nəhayət, bir çox histogram *sağa əyilmişdir* (skewed right): onlar mediandan sola nisbətən sağa daha çox uzanır. Bu, bəzi machine learning alqoritmlərinin pattern-ləri (nümunələri) aşkar etməsini bir az çətinləşdirə bilər. Sonra siz bu attribute-ları daha simmetrik və zəng formalı (bell-shaped) paylanmalara (distributions) sahib olmaq üçün transformasiya etməyi sınayacaqsınız.

İndi məşğul olduğunuz datanın növü haqqında daha yaxşı anlayışa sahib olmalısınız.

XƏBƏRDARLIQ

Dayanın! Dataya daha çox baxmazdan əvvəl, bir test set yaratmalı, onu kənara qoymalı və ona heç vaxt baxmamalısınız.

Test Set Yarat (Create a Test Set)

Bu mərhələdə datanın bir hissəsini könüllü olaraq kənara qoymaq qəribə görünə bilər. Axı, siz dataya yalnız tez bir nəzər salmısınız və hansı alqoritmlərdən istifadə edəcəyinizə qərar verməzdən əvvəl onun haqqında çoxlu şey öyrənməlisiniz, elə deyilmi? Bu doğrudur, lakin beyniniz heyranəmiz bir pattern aşkaretmə sistemidir, bu da o deməkdir ki, o, overfitting-ə çox meyillidir: əgər test set-ə baxsanız, test datasında görünüşcə maraqlı bir pattern-ə rast gələ bilərsiniz ki, bu da sizi müəyyən bir machine learning modelini seçməyə aparar. Test set-dən istifadə edərək generalization error-u qiymətləndirdiyiniz zaman, qiymətləndirməniz çox optimist olacaq və gözləniləndən o qədər də yaxşı işləməyən bir sistemi işə salacaqsınız. Buna *data snooping bias* deyilir.

Test set yaratmaq nəzəri olaraq sadədir; bəzi instance-ları təsadüfi seçin, adətən dataset-in 20%-i (dataset-iniz çox böyükdürsə daha az) və onları kənara qoyun:

```
import numpy as np
```

Sonra bu funksiyanı belə istifadə edə bilərsiniz:

```
>>> train_set, test_set = shuffle_and_split_data(housing, 0.2) >>> len(train_set) 16512 >>> len(test_set) 4128
```

Yaxşı, bu işləyir, lakin mükəmməl deyil: əgər proqramı yenidən işlətsəniz, fərqli bir test set yaradacaq! Vaxt keçdikcə siz (və ya machine learning alqoritmləriniz) bütün dataset-i görəcəksiniz ki, bu da məhz qarşısını almaq istədiyiniz şeydir.

Bir həll, test set-i ilk işlədişdə saxlamaq və sonra sonrakı işlədişlərdə onu yükləməkdir. Başqa bir seçim, `np.random.permutation()` çağırılmazdan əvvəl random number generator-un seed-ini təyin etməkdir (məsələn, `np.random.seed(42)`)⁶ ki, o, həmişə eyni qarışdırılmış indeksləri yaratsın.

Lakin bu həllərin hər ikisi növbəti dəfə yenilənmiş bir dataset əldə etdiyiniz zaman sınacaq. Dataset-i yenilədikdən sonra belə dayanıqlı bir train/test bölgüsünə (split) sahib olmaq üçün geniş yayılmış bir həll, hər instance-ın identifikatorundan (identifier) istifadə edərək onun test set-ə getməli olub-olmadığına qərar verməkdir (instance-ların unikal və dəyişməz (immutable) identifikatorları olduğunu fərz etsək). For example, hər instance-ın identifikatorunun hash-ını hesablaya və hash maksimum hash dəyərinin 20%-dən kiçik və ya bərabərdirsə həmin instance-ı test set-ə qoya bilərsiniz. Bu, dataset-i yeniləsəniz belə, test set-in çoxlu işlədiş boyunca uyğun qalmasını təmin edir. Yeni test set yeni instance-ların 20%-ni ehtiva edəcək, lakin əvvəllər training set-də olan heç bir instance-ı ehtiva etməyəcək.

Mümkün bir implementasiya budur:

```
from zlib import crc32
```

```
def is_id_in_test_set(identifier, test_ratio): return crc32(np.int64(identifier)) < test_ratio * 2**32
```

```
def split_data_with_id_hash(data, test_ratio, id_column): ids = data[id_column] in_test_set = ids.apply(lambda id_: is_id_in_test_set(id_, test_ratio)) return data.loc[~in_test_set], data.loc[in_test_set]
```

Təəssüf ki, housing dataset-inin identifikator sütunu yoxdur. Ən sadə həll sətir indeksini ID kimi istifadə etməkdir:

```
housing_with_id = housing.reset_index() # adds an index column train_set, test_set = split_data_with_id_hash(housing_with_id, 0.2, "index")
```

Əgər sətir indeksini unikal identifikator kimi istifadə edirsinizsə, yeni datanın dataset-in sonuna əlavə olunduğundan və heç bir sətirin silinmədiyindən əmin olmalısınız. Əgər bu mümkün deyilsə, onda unikal identifikator qurmaq üçün ən dayanıqlı feature-lardan istifadə etməyə cəhd edə bilərsiniz. For example, bir rayonun latitude və longitude-u bir neçə milyon il dayanıqlı qalacağına zəmanət verilir, ona görə də onları belə bir ID-yə birləşdirə bilərsiniz:⁷

```
housing_with_id["id"] = housing["longitude"] * 1000 + housing["latitude"]
train_set, test_set = split_data_with_id_hash(housing_with_id, 0.2, "id")
```

Scikit-Learn dataset-ləri müxtəlif yollarla çoxlu altçoxluğa (subsets) bölmək üçün bir neçə funksiya təqdim edir. Ən sadə funksiya `train_test_split()`-dir ki, o, demək olar ki, əvvəl təyin etdiyimiz `shuffle_and_split_data()` funksiyası ilə eyni işi görür, bir neçə əlavə funksiya ilə. Əvvəlcə, random generator seed-ini təyin etməyə imkan verən bir `random_state` parametri var. İkincisi, ona eyni sayda sətirə malik çoxlu dataset ötürə bilərsiniz və o, onları eyni indekslərdə bölər (bu çox faydalıdır, məsələn, label-lar üçün ayrı bir DataFrame-iniz varsa):

```
from sklearn.model_selection import train_test_split
```

```
train_set, test_set = train_test_split(housing, test_size=0.2, random_state=42)
```

İndiyə qədər biz tamamilə təsadüfi nümunələmə (random sampling) metodlarını nəzərdən keçirdik. Bu, dataset-iniz kifayət qədər böyükdürsə (xüsusilə attribute-ların sayına nisbətən) ümumiyyətlə yaxşıdır, lakin əgər deyilsə, əhəmiyyətli bir sampling bias daxil etmək riski ilə üzləşirsiniz. Bir sorğu şirkətindəki işçilər bir neçə sual vermək üçün 1000 nəfərə zəng etmək qərarına gəldikdə, sadəcə telefon kitabçasından təsadüfi 1000 nəfər seçmirlər. Onlar vermək istədikləri suallara münasibətdə bu 1000 nəfərin bütün əhalini təmsil etməsini təmin etməyə çalışırlar. For example, ABŞ əhalisi 51.1% qadın və 48.9% kişidir, ona görə də ABŞ-da düzgün aparılan bir sorğu nümunədə bu nisbəti saxlamağa çalışardı: 511 qadın və 489 kişi (heç olmasa cavablar cinslər arasında dəyişə bilərsə). Buna *stratified sampling* deyilir: əhali *strata* adlanan homojen altqruplara bölünür və test set-in ümumi əhalini təmsil etməsinə zəmanət vermək üçün hər stratum-dan düzgün sayda instance nümunə götürülür. Əgər sorğunu aparanlar tamamilə random sampling istifadə etsəydilər, 48.5%-dən az və ya 53.5%-dən çox qadın iştirakçısı olan əyilmiş bir test set nümunələmə ehtimalı təxminən 10.7% olardı. Hər iki halda, sorğu nəticələri çox güman ki, kifayət qədər qərəzli (biased) olardı.

Tutaq ki, bəzi ekspertlərlə söhbət etmisiniz və onlar sizə deyiblər ki, median income, median ev qiymətlərini proqnozlaşdırmaq üçün çox vacib bir attribute-dur. Test set-in bütün dataset-dəki müxtəlif income kateqoriyalarını təmsil etməsini təmin etmək istəyə bilərsiniz. Median income davamlı ədədi (numerical) attribute olduğu üçün, əvvəlcə bir income kateqoriya attribute-u yaratmalısınız. Gəlin median income histogramına daha yaxından baxaq (yenidən Şəkil 2-8-ə): median income dəyərlərinin əksəriyyəti 1.5-dən 6-ya qədər toplaşmış (yəni \$15,000-\$60,000), lakin bəzi median income-lar 6-dan çox kənara

çıxır. Dataset-inizdə hər stratum üçün kifayət qədər sayda instance olması vacibdir, əks halda bir stratum-un əhəmiyyətinin qiymətləndirilməsi qərəzli ola bilər. Bu o deməkdir ki, çox sayda stratum olmamalıdır və hər stratum kifayət qədər böyük olmalıdır. Aşağıdakı kod beş kateqoriyalı (1-dən 5-ə qədər label-lanmış) bir income kateqoriya attribute-u yaratmaq üçün `pd.cut()` funksiyasından istifadə edir; kateqoriya 1 0-dan 1.5-ə qədərdir (yəni \$15,000-dən az), kateqoriya 2 1.5-dən 3-ə qədər və s.:

```
housing["income_cat"] = pd.cut(housing["median_income"], bins=[0., 1.5, 3.0, 4.5, 6., np.inf], labels=[1, 2, 3, 4, 5])
```

Bu income kateqoriyaları Şəkil 2-9-da təmsil olunub:

```
housing["income_cat"].value_counts().sort_index().plot.bar(rot=0, grid= True )  
plt.xlabel("Income category") plt.ylabel("Number of districts") plt.show()
```

İndi income kateqoriyasına əsaslanaraq stratified sampling etməyə hazırsınız. Scikit-Learn `sklearn.model_selection` paketində dataset-inizi bir training set və bir test set-ə bölmək üçün müxtəlif strategiyalar tətbiq edən bir sıra splitter class təqdim edir. Hər splitter-in eyni datanın müxtəlif training/test bölgüləri üzərində iterator qaytaran bir `split()` metodu var.

Şəkil 2-9. Income kateqoriyalarının histogramı

Dəqiq desək, `split()` metodu datanın özünü deyil, training və test *indekslərini* verir. Çoxlu bölgüyə (`split`) sahib olmaq modelinizin performansını daha yaxşı qiymətləndirmək istəyirsinizsə faydalı ola bilər — bunu bu fəsildə daha sonra cross-validation-ı müzakirə edərkən görəcəksiniz. For example, aşağıdakı kod eyni dataset-in 10 fərqli stratified bölgüsünü yaradır:

```
from sklearn.model_selection import StratifiedShuffleSplit
```

```
splitter = StratifiedShuffleSplit(n_splits=10, test_size=0.2, random_state=42) strat_splits = []
```

```
for train_index, test_index in splitter.split(housing, housing["income_cat"]):  
    strat_train_set_n = housing.iloc[train_index] strat_test_set_n = housing.iloc[test_index]  
    strat_splits.append([strat_train_set_n, strat_test_set_n])
```

Hələlik sadəcə ilk bölgüdən istifadə edə bilərsiniz:

```
strat_train_set, strat_test_set = strat_splits[0]
```

Və ya, stratified sampling kifayət qədər geniş yayıldığı üçün, `train_test_split()` funksiyasını stratify arqumenti ilə istifadə edərək tək bir bölgü əldə etməyin daha qısa yolu var:

```
strat_train_set, strat_test_set = train_test_split(  
    housing, test_size=0.2, stratify=housing["income_cat"], random_state=42)
```



```
>>> strat_test_set["income_cat"].value_counts() / len(strat_test_set)
```

```
3 0.350533
```

```
2 0.318798
```

```
4 0.176357
```

```
5 0.114341 1 0.039971 Name: income_cat, dtype: float64
```

Bənzər kodla bütün dataset-dəki income kateqoriya nisbətlərini ölçə bilərsiniz. Şəkil 2-10 ümumi dataset-də, stratified sampling ilə yaradılan test set-də və tamamilə random sampling istifadə edərək yaradılan test set-də income kateqoriya nisbətlərini müqayisə edir. Gördüyünüz kimi, stratified sampling istifadə edərək yaradılan test set, bütün dataset-dəkilərlə demək olar ki, eyni income kateqoriya nisbətlərinə malikdir, halbuki tamamilə random sampling istifadə edərək yaradılan test set əyilmişdir (skewed).

Şəkil 2-10. Stratified-a qarşı tamamilə random sampling-in sampling bias müqayisəsi

income_cat sütununu yenidən istifadə etməyəcəksiniz, ona görə də onu sadəcə silə (drop) və datanı orijinal vəziyyətinə qaytara bilərsiniz:

```
for set_ in (strat_train_set, strat_test_set): set_.drop("income_cat", axis=1, inplace= True )
```

Biz test set yaratmağa kifayət qədər vaxt sərf etdik və yaxşı səbəbə görə: bu, çox vaxt laqeyd qalan, lakin machine learning layihəsinin kritik bir hissəsidir. Üstəlik, bu ideyaların çoxu sonradan cross-validation-ı müzakirə edərkən faydalı olacaq. İndi növbəti mərhələyə keçmək vaxtıdır: datanı araşdırmaq.

İçgörü Qazanmaq Üçün Datanı Araşdır və Vizuallaşdır

İndiyə qədər siz manipulyasiya etdiyiniz datanın növü haqqında ümumi anlayış əldə etmək üçün dataya yalnız tez bir nəzər saldınız. İndi məqsəd bir az daha dərinə getməkdir.

Əvvəlcə, test set-i kənara qoyduğunuzdan və yalnız training set-i araşdırdığınızdan əmin olun. Həmçinin, training set çox böyükdürsə, araşdırma fazasında manipulyasiyaları asan və sürətli etmək üçün bir araşdırma set-i (exploration set) nümunələmək istəyə bilərsiniz. Bu halda, training set kifayət qədər kiçikdir, ona görə də birbaşa bütün set üzərində işləyə bilərsiniz. Bütün training set-in müxtəlif transformasiyaları ilə təcrübə aparacağınız üçün, sonradan geri qayıda bilmək üçün orijinalın bir nüsxəsini çıxarmalısınız:

```
housing = strat_train_set.copy()
```

Coğrafi Datanı Vizuallaşdırmaq

Dataset coğrafi məlumatı (latitude və longitude) ehtiva etdiyi üçün, datanı vizuallaşdırmaq

məqsədlə bütün rayonların scatterplot-unu yaratmaq yaxşı ideyadır (Şəkil 2-11):

```
housing.plot(kind="scatter", x="longitude", y="latitude", grid= True ) plt.show()
```

Şəkil 2-11. Datanın coğrafi scatterplot-u

Bu, həqiqətən California-ya bənzəyir, lakin bundan başqa hər hansı xüsusi bir pattern görmək çətindir. alpha seçimini 0.2-yə təyin etmək, data nöqtələrinin sıxlığının yüksək olduğu yerləri vizuallaşdırmağı çox asanlaşdırır (Şəkil 2-12):

```
housing.plot(kind="scatter", x="longitude", y="latitude", grid= True , alpha=0.2) plt.show()
```

İndi bu daha yaxşıdır: yüksək sıxlıqlı əraziləri aydın görə bilərsiniz, yəni Bay Area və Los Angeles və San Diego ətrafı, üstəlik Central Valley-də (xüsusilə Sacramento və Fresno ətrafı) kifayət qədər yüksək sıxlıqlı ərazilərin uzun bir xətti.

Beynimiz şəkillərdəki pattern-ləri aşkar etməkdə çox yaxşıdır, lakin pattern-lərin seçilməsini təmin etmək üçün vizuallaşdırma parametrləri ilə təcrübə aparmalı ola bilərsiniz.

Şəkil 2-12. Yüksək sıxlıqlı əraziləri vurğulayan daha yaxşı bir vizuallaşdırma

Sonra, ev qiymətlərinə baxırsınız (Şəkil 2-13). Hər dairənin radiusu rayonun population-unu (s seçimi), rəng isə qiyməti (c seçimi) təmsil edir. Burada jet adlanan, mavidən (aşağı dəyərlər) qırmızıya (yüksək qiymətlər) qədər dəyişən öncədən təyin olunmuş bir rəng xəritəsi (cmap seçimi) istifadə edirsiniz:⁸

```
housing.plot(kind="scatter", x="longitude", y="latitude", grid= True ,  
s=housing["population"] / 100, label="population", c="median_house_value", cmap="jet",  
colorbar= True , legend= True , sharex= False , figsize=(10, 7)) plt.show()
```

Bu şəkil sizə deyir ki, ev qiymətləri yerləşmə (məsələn, okeana yaxınlıq) və population sıxlığı ilə çox bağlıdır, çox güman ki, bunu artıq bilirdiniz. Bir clustering alqoritmi əsas cluster-i aşkar etmək və cluster mərkəzlərinə yaxınlığı ölçən yeni feature-lar əlavə etmək üçün faydalı olardı. ocean proximity attribute-u da faydalı ola bilər, baxmayaraq ki, Şimali California-da sahilə yaxın rayonlarda ev qiymətləri o qədər də yüksək deyil, ona görə də bu sadə bir qayda deyil.

Şəkil 2-13. California ev qiymətləri: qırmızı bahalı, mavi ucuz, daha böyük dairələr daha böyük population-a malik əraziləri göstərir

Korrelyasiyaları Axtar (Look for Correlations)

Dataset çox böyük olmadığı üçün, corr() metodundan istifadə edərək hər attribute cütü arasındakı *standard correlation coefficient*-i (həmçinin *Pearson's r* adlanır) asanlıqla hesablaya bilərsiniz:

```
corr_matrix = housing.corr()
```

İndi hər attribute-un median ev dəyəri ilə nə qədər correlation əlaqəsində olduğuna baxa bilərsiniz:

```
>>> corr_matrix["median_house_value"].sort_values(ascending=False)
median_house_value 1.000000 median_income 0.688380 total_rooms 0.137455
housing_median_age 0.102175 households 0.071426 total_bedrooms 0.054635 population
-0.020153 longitude -0.050859 latitude -0.139584 Name: median_house_value, dtype:
float64
```

Correlation coefficient -1-dən 1-ə qədər dəyişir. 1-ə yaxın olduqda, güclü müsbət (positive) correlation olduğu deməkdir; for example, median income artdıqda median ev dəyəri də artmağa meyllidir. Coefficient -1-ə yaxın olduqda, güclü mənfi (negative) correlation deməkdir; latitude ilə median ev dəyəri arasında kiçik bir negative correlation görə bilərsiniz (yəni şimala getdikcə qiymətlərin bir az aşağı düşmə meylli var). Nəhayət, 0-a yaxın coefficient-lər heç bir xətti (linear) correlation olmadığı deməkdir.

Attribute-lar arasında correlation-ı yoxlamağın başqa bir yolu Pandas scatter_matrix() funksiyasından istifadə etməkdir ki, o, hər numerical attribute-u hər digər numerical attribute-a qarşı çəkir. İndi 11 numerical attribute olduğu üçün $11^2 = 121$ qrafik alardınız ki, bu da bir səhifəyə sığmazdı — ona görə də median ev dəyəri ilə ən çox correlation əlaqəsində olan kimi görünən bir neçə perspektivli attribute-a diqqət yetirmək qərarına gəlersiniz (Şəkil 2-14):

```
from pandas.plotting import scatter_matrix
```

```
attributes = ["median_house_value", "median_income", "total_rooms",
"housing_median_age"] scatter_matrix(housing[attributes], figsize=(12, 8)) plt.show()
```

Şəkil 2-14. Bu scatter matrix hər numerical attribute-u hər digər numerical attribute-a qarşı çəkir, üstəlik əsas diaqonalda (yuxarı soldan aşağı sağa) hər numerical attribute-un dəyərlərinin histogramını göstərir

Əgər Pandas hər dəyişəni özünə qarşı çəksəydi, əsas diaqonal düz xətlərlə dolu olardı ki, bu da çox faydalı olmazdı. Ona görə də əvəzinə Pandas hər attribute-un histogramını göstərir (digər seçimlər mövcuddur; daha çox detal üçün Pandas sənədlərinə baxın).

Correlation scatterplot-larına baxaraq, median ev dəyərini proqnozlaşdırmaq üçün ən perspektivli attribute median income kimi görünür, ona görə də onların scatterplot-una yaxınlaşırırsınız (Şəkil 2-15):

```
housing.plot(kind="scatter", x="median_income", y="median_house_value", alpha=0.1,
grid=True)

plt.show()
```

Şəkil 2-15. Median income-a qarşı median ev dəyəri

Bu qrafik bir neçə şeyi aşkar edir. Əvvəlcə, correlation həqiqətən kifayət qədər güclüdür; yuxarı meyli aydın görə bilərsiniz və nöqtələr o qədər də dağınıq deyil. İkincisi, əvvəl fərq etdiyiniz qiymət kəsimi (price cap) \$500,000-də üfüqi bir xətt kimi aydın görünür. Lakin qrafik həm də digər daha az aşkar düz xətləri aşkar edir: \$450,000 ətrafında üfüqi bir xətt, \$350,000 ətrafında başqa biri, bəlkə \$280,000 ətrafında biri və ondan aşağı bir neçəsi. Alqoritmlərinizin bu data qəribəliklərini (quirks) təkrar etməyi öyrənməsinin qarşısını almaq üçün müvafiq rayonları silməyi sınamaq istəyə bilərsiniz.

XƏBƏRDARLIQ

Correlation coefficient yalnız xətti (linear) correlation-ları ölçür (“x artdıqca, y ümumiyyətlə artır/azalır”). O, qeyri-xətti (nonlinear) əlaqələri tamamilə gözdən qaçıra bilər (məsələn, “x 0-a yaxınlaşdıqca, y ümumiyyətlə artır”). Şəkil 2-16 müxtəlif dataset-ləri onların correlation coefficient-i ilə birlikdə göstərir. Diqqət edin ki, alt sıradakı bütün qrafiklərin correlation coefficient-i 0-a bərabərdir, baxmayaraq ki, onların oxları açıq-aşkar müstəqil deyil: bunlar nonlinear əlaqələrin nümunələridir. Həmçinin, ikinci sıra correlation coefficient-in 1 və ya -1-ə bərabər olduğu nümunələri göstərir; diqqət edin ki, bunun mailliklə (slope) heç bir əlaqəsi yoxdur. For example, sizin inç ilə boyunuzun, fut və ya nanometr ilə boyunuzla correlation coefficient-i 1-dir.

Şəkil 2-16. Müxtəlif dataset-lərin standard correlation coefficient-i (mənbə: Wikipedia; public domain şəkil)

Attribute Kombinasiyaları ilə Təcrübə Apar

Ümid edirəm əvvəlki bölmələr sizə datanı araşdırmaq və içgörü qazanmaq üçün bir neçə üsul barədə təsəvvür verdi. Datanı machine learning alqoritminə ötürməzdən əvvəl təmizləmək istəyə biləcəyiniz bir neçə data qəribəliyini müəyyən etdiniz və attribute-lar arasında, xüsusilə target attribute ilə maraqlı correlation-lar tapdınız. Həmçinin bəzi attribute-ların sağa-əyilmiş (skewed-right) paylanmaya malik olduğunu fərq etdiniz, ona görə də onları transformasiya etmək istəyə bilərsiniz (məsələn, loqarifmlərini və ya kvadrat kökünü hesablayaraq). Əlbəttə, hər layihə ilə nəticələriniz əhəmiyyətli dərəcədə dəyişəcək, lakin ümumi ideyalar oxşardır.

Datanı machine learning alqoritmləri üçün hazırlamazdan əvvəl etmək istəyə biləcəyiniz son bir şey müxtəlif attribute kombinasiyalarını sınaqdır. For example, bir rayondakı otaqların ümumi sayı, neçə household (ev təsərrüfatı) olduğunu bilmirsinizsə o qədər də faydalı deyil. Həqiqətən istədiyiniz şey household başına otaq sayıdır. Eyni şəkildə, yataq otaqlarının (bedrooms) ümumi sayı özlüyündə o qədər də faydalı deyil: çox güman ki, onu otaqların sayı ilə müqayisə etmək istəyirsiniz. Və household başına population da baxmağa maraqlı bir attribute kombinasiyası kimi görünür. Bu yeni attribute-ları belə yaradırsınız:

```
housing["rooms_per_house"] = housing["total_rooms"] / housing["households"]  
housing["bedrooms_ratio"] = housing["total_bedrooms"] / housing["total_rooms"]
```

```
housing["people_per_house"] = housing["population"] / housing["households"]
```

Və sonra yenidən correlation matrisinə baxırsınız:

```
>>> corr_matrix = housing.corr() >>>
corr_matrix["median_house_value"].sort_values(ascending=False) median_house_value
1.000000 median_income 0.688380 rooms_per_house 0.143663 total_rooms 0.137455
housing_median_age 0.102175 households 0.071426 total_bedrooms 0.054635 population
-0.020153 people_per_house -0.038224 longitude -0.050859
latitude -0.139584 bedrooms_ratio -0.256397 Name: median_house_value, dtype: float64
```

Hey, pis deyil! Yeni bedrooms_ratio attribute-u otaqların və ya yataq otaqlarının ümumi sayından median ev dəyəri ilə daha çox correlation əlaqəsindədir. Görünür, bedroom/room nisbəti daha aşağı olan evlər daha bahalı olmağa meyllidir. Household başına otaq sayı da bir rayondakı otaqların ümumi sayından daha məlumatvericidir — açıq-aşkar, evlər nə qədər böyükdürsə, bir o qədər bahalıdır.

Bu araşdırma raundunun tamamilə hərtərəfli olması vacib deyil; məqsəd düzgün addımla başlamaq və ilk kifayət qədər yaxşı bir prototipi əldə etməyə kömək edəcək içgörüləri tez qazanmaqdır. Lakin bu iterativ bir prosesdir: prototipi işə saldıqdan sonra, daha çox içgörü qazanmaq üçün onun çıxışını təhlil edərək və bu araşdırma addımına geri qayıda bilərsiniz.

Datanı Machine Learning Alqoritmləri Üçün Hazırla

Datanı machine learning alqoritmləriniz üçün hazırlamaq vaxtıdır. Bunu əl ilə etmək əvəzinə, bir neçə yaxşı səbəbə görə bu məqsəd üçün funksiyalar yazmalısınız:

- Bu, sizə bu transformasiyaları istənilən dataset üzərində asanlıqla təkrar etməyə imkan verəcək (məsələn, növbəti dəfə təzə bir dataset əldə etdiyiniz zaman).
- Tədricən gələcək layihələrdə yenidən istifadə edə biləcəyiniz bir transformasiya funksiyaları kitabxanası quracaqsınız.
- Bu funksiyaları canlı (live) sisteminizdə yeni datanı alqoritmlərinizə ötürməzdən əvvəl transformasiya etmək üçün istifadə edə bilərsiniz.
- Bu, sizə müxtəlif transformasiyaları asanlıqla sınağa və transformasiyaların hansı kombinasiyasının ən yaxşı işlədiyini görməyə imkan verəcək.

Lakin əvvəlcə təmiz bir training set-ə qayıdın (strat_train_set-i yenidən kopyalayaraq). Həmçinin, predictor-ları və label-ları ayırmalısınız, çünki eyni transformasiyaları predictor-lara və target dəyərlərə tətbiq etmək mütləq istəmirsiniz (qeyd edin ki, drop() datanın bir nüsxəsini yaradır və strat_train_set-ə təsir etmir):

```
housing = strat_train_set.drop("median_house_value", axis=1) housing_labels =
strat_train_set["median_house_value"].copy()
```

Əksər machine learning alqoritmləri çatışmayan (missing) feature-larla işləyə bilmir, ona görə də bunlarla məşğul olmalısınız. For example, əvvəl total_bedrooms attribute-unun bəzi çatışmayan dəyərlərə malik olduğunu fərqləndirdiniz. Bunu düzəltmək üçün üç seçiminiz var:

1. Müvafiq rayonlardan qurtulun.
2. Bütün attribute-dan qurtulun.
3. Çatışmayan dəyərləri hansısa dəyərə təyin edin (sıfır, mean, median və s.). Buna *imputation* deyilir.

Bunları Pandas DataFrame-in dropna(), drop() və fillna() metodlarından istifadə edərək asanlıqla edə bilərsiniz:

```
housing.dropna(subset=["total_bedrooms"], inplace= True ) # option 1
```

```
housing.drop("total_bedrooms", axis=1) # option 2
```

```
median = housing["total_bedrooms"].median() # option 3
```

```
housing["total_bedrooms"].fillna(median, inplace= True )
```

Ən az dağıdıcı olduğu üçün seçim 3-ü seçirsiniz, lakin yuxarıdakı kod əvəzinə əlverişli bir Scikit-Learn class-ından istifadə edəcəksiniz: SimpleImputer. Üstünlüyü budur ki, o, hər feature-un median dəyərini saxlayacaq; bu, çatışmayan dəyərləri yalnız training set-də deyil, həm də validation set-də, test set-də və modelə ötürülən istənilən yeni datada impute etməyi mümkün edəcək. Ondan istifadə etmək üçün əvvəlcə bir SimpleImputer instance yaratmalı, hər attribute-un çatışmayan dəyərlərini həmin attribute-un median-ı ilə əvəz etmək istədiyinizi göstərməlisiniz:

```
from sklearn.impute import SimpleImputer
```

```
imputer = SimpleImputer(strategy="median")
```

Median yalnız numerical attribute-lar üzərində hesablana bildiyi üçün, sonra yalnız numerical attribute-lara malik datanın bir nüsxəsini yaratmalısınız (bu, mətn attribute-u olan ocean_proximity-ni istisna edəcək):

```
housing_num = housing.select_dtypes(include=[np.number])
```

İndi fit() metodundan istifadə edərək imputer instance-ını training datasına fit edə bilərsiniz:

```
imputer.fit(housing_num)
```

Imputer sadəcə hər attribute-un median-ını hesablayıb nəticəni öz statistics_ instance dəyişənində saxladı. Yalnız total_bedrooms attribute-unun çatışmayan dəyərləri var idi,

lakin sistem canlandıqdan (go live) sonra yeni datada çatışmayan dəyərlərin olmayacağına əmin ola bilməzsiniz, ona görə də imputer-i bütün numerical attribute-lara tətbiq etmək daha təhlükəsizdir:

```
>>> imputer.statistics_ array([-118.51 , 34.26 , 29. , 2125. , 434. , 1167. , 408. , 3.5385]) >>>
housing_num.median().values array([-118.51 , 34.26 , 29. , 2125. , 434. , 1167. , 408. , 3.5385])
```

İndi bu “öyrədilmiş” imputer-dən istifadə edərək çatışmayan dəyərləri öyrənilmiş median-larla əvəz edərək training set-i transformasiya edə bilərsiniz:

```
X = imputer.transform(housing_num)
```

Çatışmayan dəyərlər həmçinin mean dəyəri ilə (strategy="mean"), və ya ən tez-tez rast gəlinən (most frequent) dəyərlə (strategy="most_frequent"), və ya sabit (constant) dəyərlə (strategy="constant", fill_value=...) əvəz edilə bilər. Son iki strategiya qeyri-numerical datanı dəstəkləyir.

MƏSLƏHƏT

sklearn.impute paketində daha güclü imputer-lər də mövcuddur (hər ikisi yalnız numerical feature-lar üçün):

KNNImputer hər çatışmayan dəyəri həmin feature üçün k -nearest neighbor-ların dəyərlərinin mean-i ilə əvəz edir. Məsafə mövcud bütün feature-lara əsaslanır.

IterativeImputer çatışmayan dəyərləri bütün digər mövcud feature-lara əsaslanaraq proqnozlaşdırmaq üçün hər feature üzrə bir regression modeli öyrədir. Sonra modeli yenilənmiş data üzərində yenidən öyrədir və prosesi bir neçə dəfə təkrarlayır, hər iterasiyada modelləri və əvəzedici dəyərləri yaxşılaşdırır.

SCIKIT-LEARN DİZAYNI

Scikit-Learn-in API-si diqqətəlayiq dərəcədə yaxşı dizayn edilib. Əsas dizayn prinsipləri bunlardır:⁹

Ardıcılıq (Consistency)

Bütün obyektlər ardıcıl və sadə bir interfeysi paylaşır:

Estimator-lar

Bir dataset-ə əsaslanaraq bəzi parametrləri qiymətləndirə bilən istənilən obyekt *estimator* adlanır (məsələn, SimpleImputer bir estimator-dur). Qiymətləndirmənin özü fit() metodu ilə həyata keçirilir və o, bir dataset-i parametr kimi qəbul edir, və ya supervised learning alqoritmləri üçün iki dataset — ikinci dataset label-ları ehtiva edir. Qiymətləndirmə prosesinə rəhbərlik etmək üçün lazım olan istənilən digər parametr hyperparameter sayılır

(məsələn, SimpleImputer-in strategy-si) və o, instance dəyişəni kimi təyin edilməlidir (ümumiyyətlə konstruktor parametri vasitəsilə).

Transformer-lər

Bəzi estimator-lar (məsələn, SimpleImputer) bir dataset-i həm də transformasiya edə bilər; bunlar *transformer* adlanır. Bir daha, API sadədir: transformasiya, transformasiya ediləcək dataset-i parametr kimi qəbul edən transform() metodu ilə həyata keçirilir. O, transformasiya edilmiş dataset-i qaytarır. Bu transformasiya ümumiyyətlə öyrənilmiş parametrlərə əsaslanır, SimpleImputer halında olduğu kimi. Bütün transformer-lərin həm də fit_transform() adlı əlverişli bir metodu var ki, o, fit() və sonra transform() çağırmağa ekvivalentdir (lakin bəzən fit_transform() optimallaşdırılır və çox daha sürətli işləyir).

Predictor-lar

Nəhayət, bəzi estimator-lar, bir dataset verildikdə, proqnozlar verə bilirlər; onlar *predictor* adlanır. For example, əvvəlki fəsildəki LinearRegression modeli bir predictor idi: bir ölkənin adambaşına ÜDM-i (GDP per capita) verildikdə, o, həyatdan məmnunluğu (life satisfaction) proqnozlaşdırırdı. Bir predictor-un yeni instance-lar dataset-ini qəbul edən və müvafiq proqnozlar dataset-ini qaytaran predict() metodu var. Onun həmçinin proqnozların keyfiyyətini ölçən bir score() metodu var ki, bunun üçün bir test set (və supervised learning alqoritmləri halında müvafiq label-lar) verilir.¹⁰

İnspeksiya (Inspection)

Estimator-un bütün hyperparameter-ləri birbaşa public instance dəyişənləri vasitəsilə əlçatandır (məsələn, imputer.strategy), və estimator-un bütün öyrənilmiş parametrləri alt xətt (underscore) suffiksli public instance dəyişənləri vasitəsilə əlçatandır (məsələn, imputer.statistics_). Class-ların yayılmaması (Nonproliferation of classes)

Dataset-lər ev hazırı class-lar əvəzinə NumPy array-ləri və ya SciPy sparse matrisləri kimi təmsil olunur. Hyperparameter-lər sadəcə adi Python string-ləri və ya rəqəmlərdir.

Kompozisiya (Composition)

Mövcud tikinti blokları mümkün qədər yenidən istifadə olunur. For example, ixtiyari bir transformer ardıcılığından və ardınca son bir estimator-dan bir Pipeline estimator yaratmaq asandır, görəcəyiniz kimi. Ağıllı default-lar (Sensible defaults)

Scikit-Learn əksər parametrlər üçün məqbul default dəyərlər təqdim edir ki, bu da işləyən bir baseline sistemi tez yaratmağı asanlaşdırır.

Scikit-Learn transformer-ləri input kimi Pandas DataFrame-lər verildikdə belə NumPy array-ləri (və ya bəzən SciPy sparse matrisləri) qaytarır.¹¹ Beləliklə, imputer.transform(housing_num)-un çıxışı bir NumPy array-dir: X-in nə sütun adları, nə də indeksi var. Xoşbəxtlikdən, X-i bir DataFrame-ə bükmək (wrap) və sütun adlarını və indeksi housing_num-dan bərpa etmək o qədər də çətin deyil:


```
housing_tr = pd.DataFrame(X, columns=housing_num.columns,  
index=housing_num.index)
```

Mətn və Categorical Attribute-larla İşləmək

İndiyə qədər biz yalnız numerical attribute-larla məşğul olduq, lakin datanız mətn attribute-larını da ehtiva edə bilər. Bu dataset-də sadəcə biri var: `ocean_proximity` attribute-u. Gəlin onun ilk bir neçə instance üçün dəyərini baxaq:

```
>>> housing_cat = housing[["ocean_proximity"]] >>> housing_cat.head(8)  
ocean_proximity 13096 NEAR BAY 14973 <1H OCEAN 3785 INLAND 14689 INLAND  
20507 NEAR OCEAN 1286 INLAND 18078 <1H OCEAN 4396 NEAR BAY
```

Bu, ixtiyari mətn deyil: məhdud sayda mümkün dəyər var və onların hər biri bir kateqoriyanı təmsil edir. Beləliklə, bu attribute bir categorical attribute-dur. Əksər machine learning alqoritmləri rəqəmlərlə işləməyə üstünlük verir, ona görə də gəlin bu kateqoriyaları mətndən rəqəmlərə çevirək. Bunun üçün Scikit-Learn-in `OrdinalEncoder` class-ından istifadə edə bilərik:

```
from sklearn.preprocessing import OrdinalEncoder
```

```
ordinal_encoder = OrdinalEncoder() housing_cat_encoded =  
ordinal_encoder.fit_transform(housing_cat)
```

housing_cat_encoded-dəki ilk bir neçə kodlanmış dəyər belə görünür:

```
>>> housing_cat_encoded[:8] array([[3.], [0.], [1.], [1.], [4.], [1.], [0.], [3.]])
```

`categories_` instance dəyişənindən istifadə edərək kateqoriyaların siyahısını əldə edə bilərsiniz. O, hər categorical attribute üçün 1D array kateqoriyalar ehtiva edən bir siyahıdır (bu halda, yalnız bir categorical attribute olduğu üçün tək bir array ehtiva edən bir siyahı):

```
>>> ordinal_encoder.categories_
```

```
[array(['<1H OCEAN', 'INLAND', 'ISLAND', 'NEAR BAY', 'NEAR OCEAN'], dtype=object)]
```

Bu təmsilin bir problemi budur ki, ML alqoritmləri iki yaxın dəyərin iki uzaq dəyərdən daha oxşar olduğunu fərz edəcək. Bu bəzi hallarda yaxşı ola bilər (məsələn, “pis”, “orta”, “yaxşı” və “əla” kimi sıralanmış kateqoriyalar üçün), lakin `ocean_proximity` sütunu üçün açıq-aşkar belə deyil (məsələn, kateqoriya 0 və 4 açıq-aşkar kateqoriya 0 və 1-dən daha oxşardır). Bu problemi həll etmək üçün geniş yayılmış bir həll, hər kateqoriya üçün bir binary attribute yaratmaqdır: kateqoriya "<1H OCEAN" olduqda 1-ə bərabər bir attribute (əks halda 0), kateqoriya "INLAND" olduqda 1-ə bərabər başqa bir attribute (əks halda 0) və s. Buna *one-hot encoding* deyilir, çünki yalnız bir attribute 1-ə (qaynar, hot) bərabər olacaq, qalanları isə 0 (soyuq, cold) olacaq. Yeni attribute-lar bəzən *dummy* attribute-lar adlanır. Scikit-Learn

categorical dəyərləri one-hot vektorlara çevirmək üçün bir OneHotEncoder class təqdim edir:

```
from sklearn.preprocessing import OneHotEncoder
```

```
cat_encoder = OneHotEncoder() housing_cat_1hot =  
cat_encoder.fit_transform(housing_cat)
```

Default olaraq, OneHotEncoder-in çıxışı NumPy array əvəzinə bir SciPy *sparse matrix*-dir:

```
>>> housing_cat_1hot <16512x5 sparse matrix of type '<class 'numpy.float64'>' with 16512  
stored elements in Compressed Sparse Row format>
```

Sparse matrix, əsasən sıfırlardan ibarət matrislər üçün çox effektiv bir təmsildir. Həqiqətən, daxili olaraq o, yalnız sıfırdan fərqli dəyərləri və onların mövqelərini saxlayır. Bir categorical attribute yüzlərlə və ya minlərlə kateqoriyaya malik olduqda, onu one-hot encode etmək hər sətirdə tək bir 1 istisna olmaqla 0-larla dolu çox böyük bir matrislə nəticələnir. Bu halda, sparse matrix sizə məhz lazım olan şeydir: o, çoxlu yaddaşa qənaət edəcək və hesablamaları sürətləndirəcək. Sparse matrix-i əsasən normal bir 2D array kimi istifadə edə bilərsiniz,¹² lakin onu (sıx, dense) NumPy array-ə çevirmək istəyirsinizsə, sadəcə toarray() metodunu çağırın:

```
>>> housing_cat_1hot.toarray() array([[0., 0., 0., 1., 0.], [1., 0., 0., 0., 0.], [0., 1., 0., 0., 0.], ..., [0.,  
0., 0., 0., 1.], [1., 0., 0., 0., 0.], [0., 0., 0., 0., 1.]])
```

Alternativ olaraq, OneHotEncoder yaradarkən sparse=False təyin edə bilərsiniz, bu halda transform() metodu birbaşa adi (sıx, dense) NumPy array qaytaracaq.

OrdinalEncoder-də olduğu kimi, encoder-in categories_ instance dəyişənindən istifadə edərək kateqoriyaların siyahısını əldə edə bilərsiniz:

```
>>> cat_encoder.categories_ [array(['<1H OCEAN', 'INLAND', 'ISLAND', 'NEAR BAY',  
'NEAR OCEAN'], dtype=object)]
```

Pandas-da get_dummies() adlı bir funksiya var ki, o da hər categorical feature-u hər kateqoriya üçün bir binary feature ilə one-hot təmsilə çevirir:

```
>>> df_test = pd.DataFrame({"ocean_proximity": ["INLAND", "NEAR BAY"]}) >>>  
pd.get_dummies(df_test) ocean_proximity_INLAND ocean_proximity_NEAR BAY 0 1 0 1  
0 1
```

Yaxşı və sadə görünür, bəs niyə onu OneHotEncoder əvəzinə istifadə etməmək? Yaxşı, OneHotEncoder-in üstünlüyü budur ki, o, hansı kateqoriyalar üzərində öyrədildiyini xatırlayır. Bu çox vacibdir, çünki modeliniz production-da olduqdan sonra, ona training zamanı olduğu kimi tam eyni feature-lar ötürülməlidir: nə çox, nə az. Öyrədilmiş cat_encoder-imizə eyni df_test-i transformasiya etdirdikdə (fit_transform() yox, transform() istifadə edərək) nə verdiyinə baxın:

```
>>> cat_encoder.transform(df_test) array([[0., 1., 0., 0., 0.], [0., 0., 0., 1., 0.]])
```

Fərqi görürsünüz? `get_dummies()` yalnız iki kateqoriya gördü, ona görə də iki sütun verdi, halbuki `OneHotEncoder` hər öyrənilmiş kateqoriya üçün düzgün ardıcılıqla bir sütun verdi. Üstəlik, `get_dummies()`-ə naməlum bir kateqoriya (məsələn, "<2H OCEAN") ehtiva edən bir `DataFrame` ötürsəniz, o, məmnuniyyətlə onun üçün bir sütun yaradacaq:

```
>>> df_test_unknown = pd.DataFrame({"ocean_proximity": ["<2H OCEAN", "ISLAND"]})
>>> pd.get_dummies(df_test_unknown) ocean_proximity_<2H OCEAN
ocean_proximity_ISLAND 0 1 0 1 0 1
```

Lakin `OneHotEncoder` daha ağıllıdır: o, naməlum kateqoriyanı aşkar edəcək və bir `exception` qaldıracaq. İstəsəniz, `handle_unknown` hyperparameter-ini "ignore"-a təyin edə bilərsiniz, bu halda o, sadəcə naməlum kateqoriyanı sıfırlarla təmsil edəcək:

```
>>> cat_encoder.handle_unknown = "ignore" >>>
cat_encoder.transform(df_test_unknown) array([[0., 0., 0., 0., 0.], [0., 0., 1., 0., 0.]])
```

MƏSLƏHƏT

Əgər bir categorical attribute çoxlu mümkün kateqoriyaya malikdirsə (məsələn, ölkə kodu, peşə, növ), onda one-hot encoding çoxlu input feature ilə nəticələnəcək. Bu, training-i ləngidə və performansını pisləşdirə bilər. Bu baş verərsə, categorical input-u kateqoriyalarla bağlı faydalı numerical feature-larla əvəz etmək istəyə bilərsiniz: for example, `ocean_proximity` feature-unu okeana məsafə ilə əvəz edə bilərsiniz (eyni şəkildə, bir ölkə kodu ölkənin `population`-u və adambaşına `ÜDM`-i ilə əvəz edilə bilər). Alternativ olaraq, GitHub-da `category_encoders` paketinin təqdim etdiyi encoder-lərdən birini istifadə edə bilərsiniz. Və ya, neural network-lərlə işləyərkən, hər kateqoriyanı *embedding* adlanan öyrənilə bilən, aşağı-ölçülü (low-dimensional) bir vektorla əvəz edə bilərsiniz. Bu, *representation learning*-in bir nümunəsidir (daha çox detal üçün Fəsil 13 və 17-yə baxın).

İstənilən Scikit-Learn estimator-unu bir `DataFrame` istifadə edərək fit etdiyiniz zaman, estimator sütun adlarını `feature_names_in_` attribute-unda saxlayır. Scikit-Learn sonra bu estimator-a ötürülən istənilən `DataFrame`-in (məsələn, `transform()` və ya `predict()`-ə) eyni sütun adlarına malik olmasını təmin edir. Transformer-lər həmçinin transformer-in çıxışı ətrafında bir `DataFrame` qurmaq üçün istifadə edə biləcəyiniz bir `get_feature_names_out()` metodu təqdim edir:

```
>>> cat_encoder.feature_names_in_ array(['ocean_proximity'], dtype=object) >>>
cat_encoder.get_feature_names_out() array(['ocean_proximity_<1H OCEAN',
'ocean_proximity_INLAND', 'ocean_proximity_ISLAND', 'ocean_proximity_NEAR BAY',
'ocean_proximity_NEAR OCEAN'], dtype=object)

>>> df_output = pd.DataFrame(cat_encoder.transform(df_test_unknown),
```

```
... columns=cat_encoder.get_feature_names_out(),
```

```
... index=df_test_unknown.index)
```

```
...
```

Feature Scaling və Transformasiya

Datanıza tətbiq etməli olduğunuz ən vacib transformasiyalardan biri *feature scaling*-dir. Bir neçə istisna ilə, machine learning alqoritmləri input numerical attribute-lar çox fərqli miqyaslara (scales) malik olduqda yaxşı işləmir. Bu, housing datasında belədir: otaqların ümumi sayı təxminən 6-dan 39,320-ə qədər dəyişir, halbuki median income-lar yalnız 0-dan 15-ə qədər dəyişir. Heç bir scaling olmadan, əksər modellər median income-u laqeyd edib otaqların sayına daha çox diqqət yetirməyə meylli olacaq.

Bütün attribute-ları eyni miqyasa gətirməyin iki geniş yayılmış yolu var: *min-max scaling* və *standardization*.

XƏBƏRDARLIQ

Bütün estimator-larda olduğu kimi, scaler-ləri yalnız training datasına fit etmək vacibdir: training set-dən başqa heç nə üçün `fit()` və ya `fit_transform()` istifadə etməyin. Öyrədilmiş bir scaler-iniz olduqdan sonra, onu istənilən digər set-i — validation set, test set və yeni data daxil olmaqla — `transform()` etmək üçün istifadə edə bilərsiniz. Qeyd edin ki, training set dəyərləri həmişə müəyyən edilmiş aralığa miqyaslandırılacaq, lakin yeni data outlier-lər ehtiva edirsə, bunlar aralıqdan kənarda miqyaslandırıla bilər. Bunun qarşısını almaq istəyirsinizsə, sadəcə `clip` hyperparameter-ini `True`-ya təyin edin.

Min-max scaling (bir çox insan buna *normalization* deyir) ən sadəsidir: hər attribute üçün dəyərlər sürüşdürülür və yenidən miqyaslandırılır ki, nəticədə 0-dan 1-ə qədər dəyişsinlər. Bu, min dəyəri çıxmaqla və min ilə max arasındakı fərqə bölməklə həyata keçirilir. Scikit-Learn bunun üçün `MinMaxScaler` adlı bir transformer təqdim edir. Onun bir `feature_range` hyperparameter-i var ki, hansısa səbəbdən 0-1 istəmirsinizsə aralığı dəyişməyə imkan verir (məsələn, neural network-lər sıfır-mean input-larla ən yaxşı işləyir, ona görə də -1-dən 1-ə aralıq üstünlük verilir). İstifadəsi olduqca asandır:

```
from sklearn.preprocessing import MinMaxScaler
```

```
min_max_scaler = MinMaxScaler(feature_range=(-1, 1)) housing_num_min_max_scaled =  
min_max_scaler.fit_transform(housing_num)
```

Standardization fərqlidir: əvvəlcə mean dəyəri çıxır (beləliklə standardized dəyərlərin mean-i sıfır olur), sonra nəticəni standard deviation-a bölür (beləliklə standardized dəyərlərin standard deviation-ı 1-ə bərabər olur). Min-max scaling-dən fərqli olaraq, standardization dəyərləri müəyyən bir aralığa məhdudlaşdırmır. Lakin standardization outlier-lərdən çox daha az təsirlənir. For example, tutaq ki, bir rayonun median income-u

(səhvən) adi 0–15 əvəzinə 100-ə bərabərdir. 0–1 aralığına min-max scaling bu outlier-i 1-ə endirərdi və bütün digər dəyərləri 0–0.15-ə sıxardı, halbuki standardization o qədər təsirlənməzdi. Scikit-Learn standardization üçün StandardScaler adlı bir transformer təqdim edir:

```
from sklearn.preprocessing import StandardScaler
```

```
std_scaler = StandardScaler()
```

```
housing_num_std_scaled = std_scaler.fit_transform(housing_num)
```

MƏSLƏHƏT

Əgər bir sparse matrix-i əvvəlcə sıx (dense) matrixə çevirmədən miqyaslandırmaq istəyirsinizsə, with_mean hyperparameter-i False-a təyin edilmiş bir StandardScaler istifadə edə bilərsiniz: o, mean-i çıxmadan datanı yalnız standard deviation-a böləcək (çünki bu, sparsity-ni pozardı).

Bir feature-un paylanmasının *ağır quyruğu* (heavy tail) olduqda (yəni mean-dən uzaq dəyərlər eksponensial olaraq nadir olmadıqda), həm min-max scaling, həm də standardization əksər dəyərləri kiçik bir aralığa sıxacaq. Machine learning modelləri ümumiyyətlə bunu heç sevmir, Fəsil 4-də görəcəyiniz kimi. Ona görə də feature-u miqyaslandırmazdan əvvəl, əvvəlcə ağır quyruğu kiçiltmək üçün onu transformasiya etməlisiniz və mümkünə paylanmanı təxminən simmetrik etməlisiniz. For example, sağa ağır quyruğu olan müsbət feature-lar üçün bunu etməyin geniş yayılmış bir yolu feature-u onun kvadrat kökü ilə əvəz etməkdir (və ya feature-u 0 ilə 1 arasında bir qüvvətə yüksəltmək). Əgər feature-un həqiqətən uzun və ağır quyruğu varsa, məsələn *power law distribution*, onda feature-u onun loqarifmi ilə əvəz etmək kömək edə bilər. For example, population feature-u təxminən bir power law-a tabe olur: 10,000 sakini olan rayonlar 1,000 sakini olan rayonlardan yalnız 10 dəfə az tezdir, eksponensial olaraq daha az tez deyil. Şəkil 2-17 onun log-unu hesabladığınız zaman bu feature-un nə qədər daha yaxşı göründüyünü göstərir: o, Gaussian paylanmaya (yəni zəng formalı) çox yaxındır.

Şəkil 2-17. Bir feature-u Gaussian paylanmaya daha yaxın etmək üçün transformasiya etmək

Ağır-quyruqlu feature-larla məşğul olmağın başqa bir yanaşması feature-u *bucketizing* etməkdir. Bu, onun paylanmasını təxminən bərabər ölçülü bucket-lərə (kovalara) bölmək və hər feature dəyərini aid olduğu bucket-in indeksi ilə əvəz etmək deməkdir — income_cat feature-unu yaratmaq üçün etdiyimiz kimi (baxmayaraq ki, onu yalnız stratified sampling üçün istifadə etdik). For example, hər dəyəri onun persentili ilə əvəz edə bilərsiniz. Bərabər ölçülü bucket-lərlə bucketizing demək olar ki, uniform paylanmaya malik bir feature ilə nəticələnir, ona görə də əlavə scaling-ə ehtiyac yoxdur, və ya dəyərləri 0–1 aralığına məcbur etmək üçün sadəcə bucket-lərin sayına bölə bilərsiniz.

Bir feature multimodal paylanmaya malik olduqda (yəni iki və ya daha çox aydın zirvə, *mode* adlanır), məsələn `housing_median_age` feature-u kimi, onu bucketize etmək də faydalı ola bilər, lakin bu dəfə bucket ID-lərini numerical dəyərlər kimi deyil, kateqoriyalar kimi qəbul edərək. Bu o deməkdir ki, bucket indeksləri kodlanmalıdır, məsələn bir `OneHotEncoder` istifadə edərək (ona görə də adətən çox sayda bucket istifadə etmək istəməzsiniz). Bu yanaşma regression modelinə bu feature dəyərinin müxtəlif aralıqları üçün müxtəlif qaydaları daha asanlıqla öyrənməyə imkan verəcək. For example, bəlkə təxminən 35 il əvvəl tikilmiş evlərin dəbdən düşmüş özünəməxsus bir üslubu var və buna görə də yaşlarının tək başına təklif etdiyindən daha ucuzdur.

Multimodal paylanmaları transformasiya etməyin başqa bir yanaşması, *mode*-lərin hər biri üçün (heç olmasa əsasları üçün) `housing median age` ilə həmin xüsusi *mode* arasındakı oxşarlığı təmsil edən bir feature əlavə etməkdir. Oxşarlıq ölçüsü adətən bir *radial basis function* (RBF) istifadə edərək hesablanır — yalnız input dəyəri ilə sabit bir nöqtə arasındakı məsafədən asılı olan istənilən funksiya. Ən çox istifadə olunan RBF Gaussian RBF-dir ki, onun çıxış dəyəri input dəyəri sabit nöqtədən uzaqlaşdıqca eksponensial olaraq azalır. For example, `housing` yaşı x ilə 35 arasındakı Gaussian RBF oxşarlığı $\exp(-\gamma (x - 35)^2)$ tənliyi ilə verilir. γ (gamma) hyperparameter-i x 35-dən uzaqlaşdıqca oxşarlıq ölçüsünün nə qədər sürətlə azaldığını müəyyən edir. Scikit-Learn-in `rbf_kernel()` funksiyasından istifadə edərək, `housing median age` ilə 35 arasındakı oxşarlığı ölçən yeni bir Gaussian RBF feature yarada bilərsiniz:

```
from sklearn.metrics.pairwise import rbf_kernel
```

```
age_simil_35 = rbf_kernel(housing[["housing_median_age"]], [[35]], gamma=0.1)
```

Şəkil 2-18 bu yeni feature-u `housing median age`-in funksiyası kimi göstərir (sıx xətt). O, həmçinin daha kiçik bir gamma dəyəri istifadə etsəydiniz feature-un necə görünəcəyini göstərir. Qrafikin göstərdiyi kimi, yeni yaş oxşarlığı feature-u 35-də zirvə edir, məhz `housing median age` paylanmasındakı zirvə (spike) ətrafında: əgər bu xüsusi yaş qrupu daha aşağı qiymətlərlə yaxşı correlation əlaqəsindədirsə, bu yeni feature-un kömək etmə ehtimalı yüksəkdir.

Şəkil 2-18. Housing median age ilə 35 arasındakı oxşarlığı ölçən Gaussian RBF feature

İndiyə qədər biz yalnız input feature-lara baxdıq, lakin target dəyərlərin də transformasiya edilməsinə ehtiyac ola bilər. For example, əgər target paylanmasının ağır quyruğu varsa, target-i onun loqarifmi ilə əvəz etməyi seçə bilərsiniz. Lakin bunu etsəniz, regression modeli indi median ev dəyərinin özünü deyil, median ev dəyərinin *log*-unu proqnozlaşdıracaq. Əgər proqnozlaşdırılmış median ev dəyərini istəyirsinizsə, modelin proqnozunun eksponensialını hesablamaq olacaqsınız.

Xoşbəxtlikdən, Scikit-Learn transformer-lərinin əksəriyyətinin bir `inverse_transform()` metodu var ki, bu da onların transformasiyalarının tərsini hesablamağı asanlaşdırır. For example, aşağıdakı kod nümunəsi label-ları bir `StandardScaler` istifadə edərək necə

miqyaslandırmağı (input-lar üçün etdiyimiz kimi), sonra alınan miqyaslandırılmış label-lar üzərində sadə bir linear regression modeli öyrətməyi və onu bəzi yeni data üzərində proqnozlar vermək üçün istifadə etməyi göstərir, bu datanı öyrədilmiş scaler-in `inverse_transform()` metodundan istifadə edərək orijinal miqyasa geri çeviririk. Qeyd edin ki, biz label-ları Pandas Series-dən DataFrame-ə çeviririk, çünki `StandardScaler` 2D input-lar gözləyir. Həmçinin, bu nümunədə sadəlik üçün modeli sadəcə tək bir xam input feature (median income) üzərində öyrədirik:

```
from sklearn.linear_model import LinearRegression

target_scaler = StandardScaler()

scaled_labels = target_scaler.fit_transform(housing_labels.to_frame())

model = LinearRegression() model.fit(housing[["median_income"]], scaled_labels)
some_new_data = housing[["median_income"]].iloc[:5] # pretend this is new data

scaled_predictions = model.predict(some_new_data) predictions =
target_scaler.inverse_transform(scaled_predictions)
```

Bu yaxşı işləyir, lakin daha sadə bir seçim `TransformedTargetRegressor` istifadə etməkdir. Sadəcə onu qurmalı, ona regression modelini və label transformer-ini verməli, sonra orijinal miqyaslandırılmamış label-lardan istifadə edərək training set üzərində fit etməliyik. O, avtomatik olaraq transformer-dən istifadə edərək label-ları miqyaslandıracaq və regression modelini alınan miqyaslandırılmış label-lar üzərində öyrədəcək, məhz əvvəl etdiyimiz kimi. Sonra, proqnoz vermək istədiyimiz zaman, o, regression modelinin `predict()` metodunu çağıracaq və proqnozu istehsal etmək üçün scaler-in `inverse_transform()` metodundan istifadə edəcək:

```
from sklearn.compose import TransformedTargetRegressor

model = TransformedTargetRegressor(LinearRegression(),
transformer=StandardScaler()) model.fit(housing[["median_income"]], housing_labels)
predictions = model.predict(some_new_data)
```

Custom Transformer-lər

Scikit-Learn bir çox faydalı transformer təqdim etsə də, xüsusi (custom) transformasiyalar, təmizləmə əməliyyatları və ya müəyyən attribute-ları birləşdirmək kimi tapşırıqlar üçün özünüzkünü yazmalı olacaqsınız.

Heç bir training tələb etməyən transformasiyalar üçün, sadəcə input kimi bir NumPy array qəbul edən və transformasiya edilmiş array-i verən bir funksiya yaza bilərsiniz. For example, əvvəlki bölmədə müzakirə edildiyi kimi, ağır-quyruqlu paylanmalara malik feature-ları onların loqarifmi ilə əvəz edərək transformasiya etmək çox vaxt yaxşı ideyadır

(feature müsbət olduğunu və quyruğun sağda olduğunu fərz etsək). Gəlin bir log-transformer yaradaq və onu population feature-una tətbiq edək:

```
from sklearn.preprocessing import FunctionTransformer
```

```
log_transformer = FunctionTransformer(np.log, inverse_func=np.exp) log_pop =  
log_transformer.transform(housing[["population"]])
```

inverse_func argumenti optional-dır. O, sizə bir tərs transformasiya funksiyası göstərməyə imkan verir, məsələn transformer-inizi bir TransformedTargetRegressor-da istifadə etməyi planlaşdırırsınızsa.

Transformasiya funksiyanız əlavə argumentlər kimi hyperparameter-lər qəbul edə bilər. For example, əvvəlki kimi eyni Gaussian RBF oxşarlıq ölçüsünü hesablayan bir transformer-i belə yaradırsınız:

```
rbf_transformer = FunctionTransformer(rbf_kernel, kw_args=dict(Y=[[35.]], gamma=0.1))  
age_simil_35 = rbf_transformer.transform(housing[["housing_median_age"]])
```

Qeyd edin ki, RBF kernel üçün tərs funksiya yoxdur, çünki sabit bir nöqtədən müəyyən məsafədə həmişə iki dəyər var (0 məsafəsi istisna olmaqla). Həmçinin qeyd edin ki, rbf_kernel() feature-ları ayrı-ayrı emal etmir. Ona iki feature-lı bir array ötürsəniz, oxşarlıq ölçmək üçün 2D məsafəni (Euclidean) ölçəcək. For example, hər rayon ilə San Francisco arasındakı coğrafi oxşarlığı ölçəcək bir feature-u belə əlavə edirsiniz:

```
sf_coords = 37.7749, -122.41 sf_transformer = FunctionTransformer(rbf_kernel,  
kw_args=dict(Y=[sf_coords], gamma=0.1)) sf_simil =  
sf_transformer.transform(housing[["latitude", "longitude"]])
```

Custom transformer-lər həmçinin feature-ları birləşdirmək üçün faydalıdır. For example, input feature 0 və 1 arasındakı nisbəti hesablayan bir FunctionTransformer budur:

```
>>> ratio_transformer = FunctionTransformer(lambda X: X[:, [0]] / X[:, [1]]) >>>  
ratio_transformer.transform(np.array([[1., 2.], [3., 4.]]) array([[0.5 ], [0.75]])
```

FunctionTransformer çox əlverişlidir, lakin transformer-inizin öyrədilə bilən (trainable) olmasını, fit() metodunda bəzi parametrləri öyrənib sonradan transform() metodunda istifadə etməsini istəsəniz nə olar? Bunun üçün bir custom class yazmalısınız. Scikit-Learn duck typing-ə əsaslanır, ona görə də bu class hər hansı xüsusi bir base class-dan miras almaq (inherit) məcburiyyətində deyil. Ona yalnız üç metod lazımdır: fit() (ki, o, self qaytarmalıdır), transform() və fit_transform().

fit_transform()-u sadəcə TransformerMixin-i base class kimi əlavə etməklə pulsuz əldə edə bilərsiniz: default implementasiya sadəcə fit() və sonra transform() çağıracaq. Əgər BaseEstimator-u base class kimi əlavə etsəniz (və konstruktorunuzda *args və **kwargs istifadə etməkdən çəkinsəniz), iki əlavə metod da əldə edəcəksiniz: get_params() və set_params(). Bunlar avtomatik hyperparameter tuning üçün faydalı olacaq.

For example, StandardScaler kimi davranan bir custom transformer budur:

```
from sklearn.base import BaseEstimator, TransformerMixin from sklearn.utils.validation
import check_array, check_is_fitted
```

```
class StandardScalerClone (BaseEstimator, TransformerMixin): def init(self, with_mean=
True): # no *args or **kwargs! self.with_mean = with_mean
```

```
def fit(self, X, y= None): # y is required even though we don't use it
```

- `X = check_array(X)` *# checks that X is an array with finite float values*

```
self.mean_ = X.mean(axis=0) self.scale_ = X.std(axis=0) self.n_features_in_ = X.shape[1] #
every estimator stores this in fit() return self # always return self!
```

```
def transform(self, X): check_is_fitted(self) # looks for learned attributes (with trailing ) X =
check_array(X) assert self.n_features_in_ == X.shape[1] if self.with_mean: X = X -
self.mean_ return X / self.scale_
```

Diqqət ediləsi bir neçə şey:

- sklearn.utils.validation paketi input-ları doğrulamaq (validate) üçün istifadə edə biləcəyimiz bir neçə funksiya ehtiva edir. Sadəlik üçün bu kitabın qalan hissəsində belə test-ləri ötürəcəyik, lakin production kodunda onlar olmalıdır.
- Scikit-Learn pipeline-ları fit() metodunun X və y olmaqla iki arqumenti olmasını tələb edir, buna görə də y-dən istifadə etməsək də y=None arqumentinə ehtiyacımız var.
- Bütün Scikit-Learn estimator-ları fit() metodunda n_features_in_ təyin edir və transform() və ya predict()-ə ötürülən datanın bu sayda feature-a malik olmasını təmin edir.

fit() metodu self qaytarmalıdır.

- Bu implementasiya 100% tam deyil: bütün estimator-lar bir DataFrame verildikdə fit() metodunda feature_names_in_ təyin etməlidir. Üstəlik, bütün transformer-lər bir get_feature_names_out() metodu və transformasiyaları geri qaytarıla biləndə bir inverse_transform() metodu təqdim etməlidir. Daha çox detal üçün bu fəslin sonundakı son tapşırığa baxın.

Bir custom transformer öz implementasiyasında digər estimator-ları istifadə edə bilər (və çox vaxt edir). For example, aşağıdakı kod training datasındaki əsas cluster-ləri müəyyən etmək üçün fit() metodunda bir KMeans clusterer istifadə edən, sonra hər nümunənin hər cluster mərkəzinə nə qədər oxşar olduğunu ölçmək üçün transform() metodunda rbf_kernel() istifadə edən bir custom transformer-i nümayiş etdirir:

```
from sklearn.cluster import KMeans
```

```

class ClusterSimilarity (BaseEstimator, TransformerMixin): def init(self, n_clusters=10,
gamma=1.0, random_state= None ): self.n_clusters = n_clusters self.gamma = gamma
self.random_state = random_state

def fit(self, X, y= None , sample_weight= None ): self.kmeans_ = KMeans(self.n_clusters,
random_state=self.random_state) self.kmeans_.fit(X, sample_weight=sample_weight)
return self # always return self!

def transform(self, X):

return rbf_kernel(X, self.kmeans_.cluster_centers_, gamma=self.gamma)

def get_feature_names_out(self, names= None ):

return [f"Cluster {i} similarity" for i in range(self.n_clusters)]

```

MƏSLƏHƏT

Custom estimator-unuzun Scikit-Learn-in API-sinə hörmət edib-etmədiyini, bir instance-ı sklearn.utils.estimator_checks paketindən check_estimator()-ə ötürərək yoxlaya bilərsiniz. Tam API üçün <https://scikit-learn.org/stable/developers> ünvanına baxın.

Fəsil 9-da görəcəyiniz kimi, *k*-means datadakı cluster-ləri yerləşdirən bir clustering alqoritmidir. Neçə cluster axtardığı *n_clusters* hyperparameter-i ilə idarə olunur. Training-dən sonra cluster mərkəzləri *cluster_centers_* attribute-u vasitəsilə əlçatandır. KMeans-in *fit()* metodu optional bir *sample_weight* argumentini dəstəkləyir ki, bu da istifadəçiyə nümunələrin nisbi çəkirlərini göstərməyə imkan verir. *k*-means stoxastik (stochastic) bir alqoritmdir, yəni cluster-ləri yerləşdirmək üçün təsadüfiliyə əsaslanır, ona görə də təkrarlana bilən nəticələr istəyirsinizsə, *random_state* parametrini təyin etməlisiniz. Gördüyünüz kimi, tapşırığın mürəkkəbliyinə baxmayaraq, kod kifayət qədər sadədir. İndi gəlin bu custom transformer-i istifadə edək:

```

cluster_simil = ClusterSimilarity(n_clusters=10, gamma=1., random_state=42) similarities
= cluster_simil.fit_transform(housing[["latitude", "longitude"]],
sample_weight=housing_labels)

```

Bu kod bir ClusterSimilarity transformer yaradır, cluster-lərin sayını 10-a təyin edir. Sonra training set-dəki hər rayonun latitude və longitude-u ilə *fit_transform()* çağırır, hər rayonu median ev dəyərinə görə çəkiləndirir. Transformer cluster-ləri yerləşdirmək üçün *k*-means istifadə edir, sonra hər rayon ilə bütün 10 cluster mərkəzi arasındakı Gaussian RBF oxşarlığını ölçür. Nəticə hər rayon üçün bir sətir və hər cluster üçün bir sütun olan bir matrisdir. Gəlin ilk üç sətirə baxaq, iki onluq yerə yuvarlaqlaşdıraraq:

```

>>> similarities[:3].round(2) array([[0. , 0.14, 0. , 0. , 0. , 0.08, 0. , 0.99, 0. , 0.6 ], [0.63, 0. , 0.99,
0. , 0. , 0. , 0.04, 0. , 0.11, 0. ], [0. , 0.29, 0. , 0. , 0.01, 0.44, 0. , 0.7 , 0. , 0.3 ]])

```

Şəkil 2-19 *k*-means tərəfindən tapılan 10 cluster mərkəzini göstərir. Rayonlar ən yaxın cluster mərkəzinə coğrafi oxşarlıqlarına görə rənglənilib. Gördüyünüz kimi, əksər cluster-lər yüksək məskunlaşmış və bahalı ərazilərdə yerləşir.

Şəkil 2-19. Ən yaxın cluster mərkəzinə Gaussian RBF oxşarlığı

Transformasiya Pipeline-ları

Gördüyünüz kimi, düzgün ardıcılıqla icra edilməli olan bir çox data transformasiya addımı var. Xoşbəxtlikdən, Scikit-Learn belə transformasiya ardıcılıqlarına kömək etmək üçün Pipeline class-ını təqdim edir. Numerical attribute-lar üçün kiçik bir pipeline budur ki, o, əvvəlcə input feature-ları impute, sonra scale edəcək:

```
from sklearn.pipeline import Pipeline
```

```
num_pipeline = Pipeline([ ("impute", SimpleImputer(strategy="median")),  
                           ("standardize", StandardScaler()), ])
```

Pipeline konstrukturu bir addım ardıcılığını təyin edən ad/estimator cütlərinin (2-tuple) siyahısını qəbul edir. Adlar unikal olduqları və ikiqat alt xətt (__) ehtiva etmədikləri müddətcə istədiyiniz hər şey ola bilər. Onlar sonradan hyperparameter tuning-i müzakirə edərkən faydalı olacaq. Estimator-ların hamısı transformer olmalıdır (yəni `fit_transform()` metoduna malik olmalıdırlar), sonuncu istisna olmaqla — o, istənilən şey ola bilər: bir transformer, bir predictor və ya hər hansı digər estimator növü.

MƏSLƏHƏT

Jupyter notebook-da, əgər `sklearn` import edib `sklearn.set_config(display="diagram")` işlədirsə, bütün Scikit-Learn estimator-ları interaktiv diaqramlar kimi göstəriləcək. Bu, xüsusilə pipeline-ları vizuallaşdırmaq üçün faydalıdır. `num_pipeline`-ı vizuallaşdırmaq üçün, son sətiri `num_pipeline` olan bir hüceyrə işlədin. Bir estimator-a klik etmək daha çox detal göstərəcək.

Transformer-ləri adlandırmaq istəmirsinizsə, əvəzinə `make_pipeline()` funksiyasından istifadə edə bilərsiniz; o, transformer-ləri pozisional arqumentlər kimi qəbul edir və transformer-lərin class adlarından istifadə edərək (kiçik hərflərlə və alt xətt olmadan, məsələn "simpleimputer") bir Pipeline yaradır:

```
from sklearn.pipeline import make_pipeline
```

```
num_pipeline = make_pipeline(SimpleImputer(strategy="median"), StandardScaler())
```

Əgər çoxlu transformer eyni ada malikdirsə, onların adlarına bir indeks əlavə olunur (məsələn, "foo-1", "foo-2" və s.).

Pipeline-ın `fit()` metodunu çağırdığınız zaman, o, bütün transformer-lər üzərində ardıcıl olaraq `fit_transform()` çağırır, hər çağırışın çıxışını növbəti çağırışa parametr kimi ötürür,

son estimator-a çatana qədər — onun üçün isə sadəcə `fit()` metodunu çağırır.

Pipeline son estimator ilə eyni metodları təqdim edir. Bu nümunədə son estimator bir `StandardScaler`-dir, o da bir transformer-dir, ona görə də pipeline da bir transformer kimi davranır. Pipeline-in `transform()` metodunu çağırırsınız, o, bütün transformasiyaları dataya ardıcıl olaraq tətbiq edəcək. Əgər son estimator transformer əvəzinə bir predictor olsaydı, onda pipeline-in `transform()` metodu əvəzinə `predict()` metodu olardı. Onu çağırmaq bütün transformasiyaları dataya ardıcıl tətbiq edər və nəticəni predictor-un `predict()` metoduna ötürərdi.

Gəlin pipeline-in `fit_transform()` metodunu çağıraraq və çıxışın ilk iki sətirinə baxaraq, iki onluq yerə yuvarlaqlaşdıraraq:

```
>>> housing_num_prepared = num_pipeline.fit_transform(housing_num) >>>
housing_num_prepared[:2].round(2) array([[ -1.42,  1.01,  1.86,  0.31,  1.37,  0.14,  1.39, -0.94], [
  0.6 , -0.7 ,  0.91, -0.31, -0.44, -0.69, -0.37,  1.17]])
```

Əvvəl gördüyünüz kimi, gözəl bir `DataFrame` bərpa etmək istəyirsinizsə, pipeline-in `get_feature_names_out()` metodundan istifadə edə bilərsiniz:

```
df_housing_num_prepared = pd.DataFrame( housing_num_prepared,
columns=num_pipeline.get_feature_names_out(), index=housing_num.index)
```

Pipeline-lar indeksləməni (indexing) dəstəkləyir; for example, `pipeline[1]` pipeline-dakı ikinci estimator-u qaytarır, və `pipeline[:-1]` sonuncudan başqa bütün estimator-ları ehtiva edən bir Pipeline obyektini qaytarır. Estimator-lara həmçinin `steps` attribute-u vasitəsilə (ad/estimator cütlərinin siyahısı) və ya `named_steps` lüğət (dictionary) attribute-u vasitəsilə (adları estimator-lara xəritələyir) daxil ola bilərsiniz. For example, `num_pipeline["simpleimputer"]` "simpleimputer" adlı estimator-u qaytarır.

İndiyə qədər biz categorical sütunları və numerical sütunları ayrı-ayrı emal etdik. Bütün sütunları emal edə bilən, hər sütuna uyğun transformasiyaları tətbiq edən tək bir transformer-ə sahib olmaq daha əlverişli olardı. Bunun üçün bir `ColumnTransformer` istifadə edə bilərsiniz. For example, aşağıdakı `ColumnTransformer` `num_pipeline`-ı (indica təyin etdiyimiz) numerical attribute-lara və `cat_pipeline`-ı categorical attribute-a tətbiq edəcək:

```
from sklearn.compose import ColumnTransformer
```

```
num_attribs = ["longitude", "latitude", "housing_median_age", "total_rooms",
"total_bedrooms", "population", "households", "median_income"] cat_attribs =
["ocean_proximity"]
```

```
cat_pipeline = make_pipeline( SimpleImputer(strategy="most_frequent"),
OneHotEncoder(handle_unknown="ignore"))
```

```
preprocessing = ColumnTransformer([ ("num", num_pipeline, num_attribs), ("cat",
cat_pipeline, cat_attribs), ])
```

Əvvəlcə ColumnTransformer class-ını import edirik, sonra numerical və categorical sütun adlarının siyahısını təyin edib categorical attribute-lar üçün sadə bir pipeline qururuq. Nəhayət, bir ColumnTransformer qururuq. Onun konstrukturu üçlüklərin (3-tuple) bir siyahısını tələb edir, hər biri bir ad (unikal olmalı və ikiqat alt xətt ehtiva etməməlidir), bir transformer və transformer-in tətbiq olunmalı olduğu sütunların adlarının (və ya indekslərinin) siyahısını ehtiva edir.

MƏSLƏHƏT

Bir transformer istifadə etmək əvəzinə, sütunların silinməsini istəyirsinizsə "drop" string-ini, və ya sütunların toxunulmadan saxlanılmasını istəyirsinizsə "passthrough"-u göstərə bilərsiniz. Default olaraq, qalan sütunlar (yəni sadalanmayanlar) silinəcək, lakin bu sütunların fərqli şəkildə emal olunmasını istəyirsinizsə, remainder hyperparameter-ini istənilən transformer-ə (və ya "passthrough"-a) təyin edə bilərsiniz.

Bütün sütun adlarını sadalamaq o qədər də əlverişli olmadığı üçün, Scikit-Learn bir make_column_selector() funksiyası təqdim edir ki, o, müəyyən bir tipin (məsələn, numerical və ya categorical) bütün feature-larını avtomatik seçmək üçün istifadə edə biləcəyiniz bir selector funksiyası qaytarır. Bu selector funksiyasını sütun adları və ya indeksləri əvəzinə ColumnTransformer-ə ötürə bilərsiniz. Üstəlik, transformer-ləri adlandırmağa əhəmiyyət vermirsinizsə, make_column_transformer() istifadə edə bilərsiniz ki, o, make_pipeline()-ın etdiyi kimi adları sizin üçün seçir. For example, aşağıdakı kod əvvəlki ilə eyni ColumnTransformer-i yaradır, yalnız transformer-lər "num" və "cat" əvəzinə avtomatik olaraq "pipeline-1" və "pipeline-2" adlandırılır:

```
from sklearn.compose import make_column_selector, make_column_transformer

preprocessing = make_column_transformer(

(num_pipeline, make_column_selector(dtype_include=np.number)),

(cat_pipeline, make_column_selector(dtype_include=object)),)
```

İndi bu ColumnTransformer-i housing datasına tətbiq etməyə hazırıq:

```
housing_prepared = preprocessing.fit_transform(housing)
```

Əla! Bütün training dataset-ini qəbul edən və hər transformer-i uyğun sütunlara tətbiq edən, sonra transformasiya edilmiş sütunları üfüqi olaraq birləşdirən (transformer-lər heç vaxt sətirlərin sayını dəyişdirməməlidir) bir preprocessing pipeline-ımız var. Bir daha bu, bir NumPy array qaytarır, lakin preprocessing.get_feature_names_out() istifadə edərək sütun adlarını əldə edə və datanı əvvəl etdiyimiz kimi gözəl bir DataFrame-ə bükə bilərsiniz.

QEYD

OneHotEncoder bir sparse matrix qaytarır və num_pipeline bir dense matrix qaytarır. Belə bir sparse və dense matrislərin qarışığı olduqda, ColumnTransformer son matrisin sıxlığını (yəni sıfırdan fərqli hüceyrələrin nisbətini) qiymətləndirir və sıxlıq verilmiş bir həddən aşağıdırsa bir sparse matrix qaytarır (default olaraq, sparse_threshold=0.3). Bu nümunədə o, bir dense matrix qaytarır.

Layihəniz həqiqətən yaxşı gedir və bəzi modelləri öyrətməyə demək olar ki, hazırsınız! İndi indiyə qədər təcrübə apardığınız bütün transformasiyaları həyata keçirəcək tək bir pipeline yaratmaq istəyirsiniz. Gəlin pipeline-in nə edəcəyini və niyə etdiyini xülasə edək:

- Numerical feature-lardakı çatışmayan dəyərlər median ilə əvəz edilərək impute olunacaq, çünki əksər ML alqoritmləri çatışmayan dəyərlər gözləmir. Categorical feature-larda çatışmayan dəyərlər ən tez-tez rast gəlinən (most frequent) kateqoriya ilə əvəz olunacaq.
- Categorical feature one-hot encode olunacaq, çünki əksər ML alqoritmləri yalnız numerical input-ları qəbul edir.
- Bir neçə ratio feature hesablanıb əlavə olunacaq: bedrooms_ratio, rooms_per_house və people_per_house. Ümid edirik ki, bunlar median ev dəyəri ilə daha yaxşı correlation əlaqəsində olacaq və beləliklə ML modellərinə kömək edəcək.
- Bir neçə cluster similarity feature də əlavə olunacaq. Bunlar çox güman ki, modelə latitude və longitude-dan daha faydalı olacaq.
- Uzun quyruğa malik feature-lar onların loqarifmi ilə əvəz olunacaq, çünki əksər modellər təxminən uniform və ya Gaussian paylanmalara malik feature-lara üstünlük verir.
- Bütün numerical feature-lar standardize olunacaq, çünki əksər ML alqoritmləri bütün feature-lar təxminən eyni miqyasa malik olduqda üstünlük verir.

Bütün bunları edən pipeline-ı quran kod indi sizə tanış görünməlidir:

```
def column_ratio(X): return X[:, [0]] / X[:, [1]]
```

```
def ratio_name(function_transformer, feature_names_in): return ["ratio"] # feature names out
```

```
def ratio_pipeline(): return make_pipeline( SimpleImputer(strategy="median"),  
FunctionTransformer(column_ratio, feature_names_out=ratio_name), StandardScaler())
```

```
log_pipeline = make_pipeline( SimpleImputer(strategy="median"),  
FunctionTransformer(np.log, feature_names_out="one-to-one"), StandardScaler())
```

```
cluster_simil = ClusterSimilarity(n_clusters=10, gamma=1., random_state=42)
```

```
default_num_pipeline = make_pipeline(SimpleImputer(strategy="median"),  
StandardScaler()) preprocessing = ColumnTransformer([ ("bedrooms", ratio_pipeline(),
```

```
[["total_bedrooms", "total_rooms"]], ("rooms_per_house", ratio_pipeline(), ["total_rooms", "households"]], ("people_per_house", ratio_pipeline(), ["population", "households"]], ("log", log_pipeline, ["total_bedrooms", "total_rooms", "population", "households", "median_income"]],
```

```
("geo", cluster_simil, ["latitude", "longitude"]],
```

```
("cat", cat_pipeline, make_column_selector(dtype_include=object)), ],  
remainder=default_num_pipeline) # one column remaining: housing_median_age
```

Bu ColumnTransformer-i işlətsəniz, o, bütün transformasiyaları yerinə yetirir və 24 feature-lı bir NumPy array verir:

```
>>> housing_prepared = preprocessing.fit_transform(housing) >>>
```

```
housing_prepared.shape (16512, 24)
```

```
>>> preprocessing.get_feature_names_out() array(['bedrooms__ratio',  
'rooms_per_house__ratio',
```

- 'people_per_house__ratio', 'log__total_bedrooms',
- 'log__total_rooms', 'log__population', 'log__households',
- 'log__median_income', 'geo__Cluster 0 similarity', [...],
- 'geo__Cluster 9 similarity', 'cat__ocean_proximity_<1H OCEAN',
- 'cat__ocean_proximity_INLAND', 'cat__ocean_proximity_ISLAND',
- 'cat__ocean_proximity_NEAR BAY', 'cat__ocean_proximity_NEAR OCEAN',
- 'remainder__housing_median_age'], dtype=object)

Bir Model Seç və Öyrət (Select and Train a Model)

Nəhayət! Siz problemi çərçivələdiniz, datanı əldə edib araşdırdınız, bir training set və bir test set nümunələdiniz və datanızı machine learning alqoritmləri üçün avtomatik olaraq təmizləyib hazırlamaq üçün bir preprocessing pipeline yazdınız. İndi bir machine learning modeli seçməyə və öyrətməyə hazırsınız.

Training Set Üzərində Öyrət və Qiymətləndir

Yaxşı xəbər budur ki, bütün bu əvvəlki addımlar sayəsində indi işlər asanlaşacaq! Başlamaq üçün çox sadə bir linear regression modeli öyrətmək qərarına gə bilərsiniz:

```
from sklearn.linear_model import LinearRegression
```

```
lin_reg = make_pipeline(preprocessing, LinearRegression()) lin_reg.fit(housing,  
housing_labels)
```

Hazırdır! İndi işləyən bir linear regression modeliniz var. Onu training set üzərində sınayırsınız, ilk beş proqnoza baxıb onları label-larla müqayisə edirsiniz:

```
>>> housing_predictions = lin_reg.predict(housing) >>>
housing_predictions[:5].round(-2) # -2 = rounded to the nearest hundred array([243700.,
372400., 128800., 94400., 328300.]) >>> housing_labels.iloc[:5].values array([458300.,
483800., 101700., 96100., 361800.])
```

Yaxşı, işləyir, lakin həmişə yox: ilk proqnoz çox uzaqdır (\$200,000-dən çox!), digər proqnozlar isə daha yaxşıdır: ikisi təxminən 25%, ikisi isə 10%-dən az uzaqdır. Xatırlayın ki, performance measure kimi RMSE istifadə etməyi seçdiniz, ona görə də bu regression modelinin RMSE-sini bütün training set üzərində Scikit-Learn-in `mean_squared_error()` funksiyasından istifadə edərək, `squared argumenti False`-a təyin edilmiş halda ölçmək istəyirsiniz:

```
>>> from sklearn.metrics import mean_squared_error >>> lin_rmse =
mean_squared_error(housing_labels, housing_predictions, ... squared= False ) ... >>>
lin_rmse 68687.89176589991
```

Bu, heç nədən yaxşıdır, lakin açıq-aşkar əla bir bal deyil: əksər rayonların `median_housing_value`-ləri \$120,000 ilə \$265,000 arasında dəyişir, ona görə də \$68,628 tipik proqnoz səhvi həqiqətən o qədər də qənaətbəxş deyil. Bu, bir modelin training datasına underfitting etməsinin bir nümunəsidir. Bu baş verdikdə, o, feature-ların yaxşı proqnozlar vermək üçün kifayət qədər məlumat təmin etmədiyini, və ya modelin kifayət qədər güclü olmadığını ifadə edə bilər. Əvvəlki fəsildə gördüyümüz kimi, underfitting-i düzəltməyin əsas yolları daha güclü bir model seçmək, training alqoritminə daha yaxşı feature-lar vermək, və ya model üzərindəki məhdudiyyətləri azaltmaqdır. Bu model regularize olunmayıb (not regularized), bu da son seçimi aradan qaldırır. Daha çox feature əlavə etməyə cəhd edə bilərsiniz, lakin əvvəlcə necə işlədiyini görmək üçün daha mürəkkəb bir model sınamaq istəyirsiniz.

Bir `DecisionTreeRegressor` sınamaq qərarına gəlməlisiniz, çünki bu, datadakı mürəkkəb nonlinear əlaqələri tapa bilən kifayət qədər güclü bir modeldir (decision tree-lər Fəsil 6-da daha ətraflı təqdim olunur):

```
from sklearn.tree import DecisionTreeRegressor

tree_reg = make_pipeline(preprocessing, DecisionTreeRegressor(random_state=42))
tree_reg.fit(housing, housing_labels)
```

İndi model öyrədildikdən sonra onu training set üzərində qiymətləndirirsiniz:

```
>>> housing_predictions = tree_reg.predict(housing) >>> tree_rmse =
mean_squared_error(housing_labels, housing_predictions, ... squared= False )

... >>> tree_rmse 0.0
```


Dayanın, nə!? Heç bir səhv yoxdur? Bu model həqiqətən tamamilə mükəmməl ola bilərmi? Əlbəttə, modelin datanı pis şəkildə overfit etməsi daha çox ehtimaldır. Bunu necə əmin ola bilərsiniz? Əvvəl gördüyünüz kimi, əmin olduğunuz bir modeli işə salmağa hazır olana qədər test set-ə toxunmaq istəmirsiniz, ona görə də training set-in bir hissəsini training üçün, bir hissəsini isə model validasiyası üçün istifadə etməlisiniz.

Cross-Validation İstifadə Edərək Daha Yaxşı Qiymətləndirmə

Decision tree modelini qiymətləndirməyin bir yolu, training set-i daha kiçik bir training set və bir validation set-ə bölmək üçün `train_test_split()` funksiyasından istifadə etmək, sonra modellərinizi daha kiçik training set-ə qarşı öyrətmək və onları validation set-ə qarşı qiymətləndirmək olardı. Bu bir az səy tələb edir, lakin çox da çətin deyil və kifayət qədər yaxşı işləyirdi.

Əla bir alternativ Scikit-Learn-in `k-fold cross-validation_` funksiyasından istifadə etməkdir. Aşağıdakı kod training set-i *fold* adlanan 10 üst-üstə düşməyən (nonoverlapping) altçoxluğa təsadüfi olaraq bölür, sonra decision tree modelini 10 dəfə öyrədir və qiymətləndirir, hər dəfə qiymətləndirmə üçün fərqli bir fold seçir və digər 9 fold-u training üçün istifadə edir. Nəticə 10 qiymətləndirmə balını ehtiva edən bir array-dir:

```
from sklearn.model_selection import cross_val_score
```

```
tree_rmse = -cross_val_score(tree_reg, housing, housing_labels,  
scoring="neg_root_mean_squared_error", cv=10)
```

XƏBƏRDARLIQ

Scikit-Learn-in cross-validation funksiyaları cost function (aşağı daha yaxşıdır) əvəzinə utility function (böyük daha yaxşıdır) gözləyir, ona görə də scoring funksiyası əslində RMSE-nin əksidir. Bu, mənfi bir dəyərdir, ona görə də RMSE ballarını əldə etmək üçün çıxışın işarəsini dəyişməlisiniz.

Gəlin nəticələrə baxaq:

```
>>> pd.Series(tree_rmse).describe() count 10.000000 mean 66868.027288 std  
2060.966425 min 63649.536493 25% 65338.078316 50% 66801.953094 75% 68229.934454  
max 70094.778246 dtype: float64
```

İndi decision tree əvvəl göründüyü qədər yaxşı görünür. Əslində, demək olar ki, linear regression modeli qədər zəif işləyir kimi görünür! Diqqət edin ki, cross-validation sizə yalnız modelinizin performansının qiymətləndirməsini deyil, həm də bu qiymətləndirmənin nə qədər dəqiq olduğunun (yəni onun standard deviation-ının) ölçüsünü əldə etməyə imkan verir. Decision tree-nin RMSE-si təxminən 66,868-dir, standard deviation-ı isə təxminən 2,061-dir. Yalnız bir validation set istifadə etsəydiniz bu

məlumatə sahib olmazdınız. Lakin cross-validation modeli bir neçə dəfə öyrətmək hesabına başa gəlir, ona görə də həmişə mümkün deyil.

Linear regression modeli üçün eyni metriki hesablasanız, mean RMSE-nin 69,858 və standard deviation-ın 4,182 olduğunu görəcəksiniz. Beləliklə, decision tree modeli linear modeldən çox az daha yaxşı işləyir kimi görünür, lakin ciddi overfitting səbəbindən fərq minimaldır. Overfitting problemi olduğunu bilirik, çünki training error aşağıdır (əslində sıfır), validation error isə yüksəkdir.

Gəlin indi son bir model sınayaq: RandomForestRegressor. Fəsil 7-də görəcəyiniz kimi, random forest-lər feature-ların təsadüfi altçoxluqları üzərində çoxlu decision tree öyrətməklə, sonra onların proqnozlarının ortalamasını almaqla işləyir. Bu cür çoxlu digər modeldən ibarət modellər *ensemble* adlanır: onlar əsas modelin (bu halda decision tree-lərin) performansını artırır bilirlər. Kod əvvəlki ilə demək olar ki, eynidir:

```
from sklearn.ensemble import RandomForestRegressor
```

```
forest_reg = make_pipeline(preprocessing, RandomForestRegressor(random_state=42))
forest_rmse = -cross_val_score(forest_reg, housing, housing_labels,
                                scoring="neg_root_mean_squared_error", cv=10)
```

Gəlin ballara baxaq:

```
>>> pd.Series(forest_rmse).describe() count 10.000000 mean 47019.561281 std
1033.957120 min 45458.112527 25% 46464.031184 50% 46967.596354
75% 47325.694987 max 49243.765795 dtype: float64
```

Vau, bu çox daha yaxşıdır: random forest-lər bu tapşırıq üçün həqiqətən çox perspektivli görünür! Lakin, bir RandomForest öyrədib RMSE-ni training set üzərində ölçsəniz, təxminən 17,474 tapacaqsınız: bu çox daha aşağıdır, yəni hələ də kifayət qədər çox overfitting baş verir. Mümkün həllər modeli sadələşdirmək, onu məhdudlaşdırmaq (yəni regularize etmək), və ya çox daha çox training datası əldə etməkdir. Lakin random forest-lərə daha dərinə dalmazdan əvvəl, hyperparameter-ləri tənzimləməyə çox vaxt sərf etmədən machine learning alqoritmlərinin müxtəlif kateqoriyalarından bir çox digər modeli sınaqlırsınız (məsələn, müxtəlif kernel-lərlə bir neçə support vector machine və ola bilsin bir neural network). Məqsəd bir neçə (iki-beş) perspektivli modeli qısa siyahıya (shortlist) salmaqdır.

Modelini Fine-Tune Et

Tutaq ki, indi perspektivli modellərin qısa siyahısına (shortlist) sahibsiniz. İndi onları fine-tune etməlisiniz. Gəlin bunu etməyin bir neçə yoluna baxaq.

Grid Search

Bir seçim, hyperparameter dəyərlərinin əla bir kombinasiyasını tapana qədər hyperparameter-lərlə əl ilə oynamaq olardı. Bu, çox yorucu bir iş olardı və çoxlu kombinasiyanı araşdırmağa vaxtınız olmaya bilər.

Əvəzinə, sizin üçün axtarış aparmaq üçün Scikit-Learn-in GridSearchCV class-ından istifadə edə bilərsiniz. Yalnız ona hansı hyperparameter-lərlə təcrübə aparmasını və hansı dəyərləri sınamasını söyləməlisiniz və o, hyperparameter dəyərlərinin bütün mümkün kombinasiyalarını qiymətləndirmək üçün cross-validation istifadə edəcək. For example, aşağıdakı kod RandomForestRegressor üçün hyperparameter dəyərlərinin ən yaxşı kombinasiyasını axtarır:

```
from sklearn.model_selection import GridSearchCV
```

```
full_pipeline = Pipeline([ ("preprocessing", preprocessing), ("random_forest",
RandomForestRegressor(random_state=42)), ]) param_grid = [
{'preprocessing__geo__n_clusters': [5, 8, 10], 'random_forest__max_features': [4, 6, 8]},
{'preprocessing__geo__n_clusters': [10, 15], 'random_forest__max_features': [6, 8, 10]}, ]
grid_search = GridSearchCV(full_pipeline, param_grid, cv=3,
scoring='neg_root_mean_squared_error') grid_search.fit(housing, housing_labels)
```

Diqqət edin ki, bir pipeline-dakı istənilən estimator-un istənilən hyperparameter-inə istinad edə bilərsiniz, hətta bu estimator bir neçə pipeline və column transformer-in dərinliyində yuvalanmış olsa belə. For example, Scikit-Learn

"preprocessing__geo__n_clusters"-ı gördükdə, bu string-i ikiqat alt xətlərdə bölür, sonra pipeline-da "preprocessing" adlı bir estimator axtarır və preprocessing ColumnTransformer-i tapır. Sonra, bu ColumnTransformer-in içində "geo" adlı bir transformer axtarır və latitude və longitude attribute-larında istifadə etdiyimiz ClusterSimilarity transformer-i tapır. Sonra bu transformer-in n_clusters hyperparameter-ini tapır. Eyni şəkildə, random_forest__max_features "random_forest" adlı estimator-un max_features hyperparameter-inə istinad edir, ki, o da əlbəttə RandomForest modelidir (max_features hyperparameter-i Fəsil 7-də izah olunacaq).

MƏSLƏHƏT

Preprocessing addımlarını bir Scikit-Learn pipeline-ına bükmək, preprocessing hyperparameter-lərini model hyperparameter-ləri ilə birlikdə tənzimləməyə imkan verir. Bu yaxşı bir şeydir, çünki onlar çox vaxt qarşılıqlı təsir göstərir. For example, bəlkə n_clusters-i artırmaq max_features-i də artırmağı tələb edir. Əgər pipeline transformer-lərini fit etmək hesablama baxımından bahalıdırsa, pipeline-ın memory hyperparameter-ini bir caching qovluğunun yoluna təyin edə bilərsiniz: pipeline-ı ilk dəfə fit etdiyiniz zaman, Scikit-Learn fit edilmiş transformer-ləri bu qovluğa saxlayacaq. Əgər sonra

pipeline-ı eyni hyperparameter-lərlə yenidən fit etsəniz, Scikit-Learn sadəcə caching edilmiş transformer-ləri yükləyəcək.

Bu param_grid-də iki lüğət (dictionary) var, ona görə də GridSearchCV əvvəlcə birinci dict-də göstərilən n_clusters və max_features hyperparameter dəyərlərinin bütün $3 \times 3 = 9$ kombinasiyasını qiymətləndirəcək, sonra ikinci dict-dəki hyperparameter dəyərlərinin bütün $2 \times 3 = 6$ kombinasiyasını sınayacaq. Beləliklə, ümumilikdə grid search hyperparameter dəyərlərinin $9 + 6 = 15$ kombinasiyasını araşdıracaq və 3-fold cross-validation istifadə etdiyimiz üçün hər kombinasiya üçün pipeline-ı 3 dəfə öyrədəcək. Bu o deməkdir ki, ümumilikdə $15 \times 3 = 45$ raund training olacaq! Bir az vaxt apara bilər, lakin bitdikdə parametrlərin ən yaxşı kombinasiyasını belə əldə edə bilərsiniz:

```
>>> grid_search.best_params_ {'preprocessing__geo__n_clusters': 15,
'random_forest__max_features': 6}
```

Bu nümunədə, ən yaxşı model n_clusters-i 15-ə və max_features-i 8-ə təyin etməklə əldə olunur.

MƏSLƏHƏT

15, n_clusters üçün qiymətləndirilən maksimum dəyər olduğundan, çox güman ki, daha yüksək dəyərlərlə yenidən axtarış aparmalısınız; bal yaxşılaşmağa davam edə bilər.

Ən yaxşı estimator-a grid_search.best_estimator_ istifadə edərək daxil ola bilərsiniz. Əgər GridSearchCV refit=True ilə inisiallaşdırılıbsa (ki, bu default-dur), onda cross-validation istifadə edərək ən yaxşı estimator-u tapdıqdan sonra, o, onu bütün training set üzərində yenidən öyrədir. Bu adətən yaxşı ideyadır, çünki ona daha çox data vermək çox güman ki, onun performansını yaxşılaşdıracaq.

Qiymətləndirmə balları grid_search.cv_results_ istifadə edərək əlçatandır. Bu bir lüğətdir, lakin onu bir DataFrame-ə buksəniz, hər hyperparameter kombinasiyası və hər cross-validation bölgüsü üçün bütün test ballarının, həmçinin bütün bölgülər üzrə mean test balının gözəl bir siyahısını əldə edirsiniz:

```
>>> cv_res = pd.DataFrame(grid_search.cv_results_) >>>
cv_res.sort_values(by="mean_test_score", ascending= False , inplace= True ) >>> [...] #
change column names to fit on this page, and show rmse = -score >>> cv_res.head() # note:
the 1st column is the row ID n_clusters max_features split0 split1 split2 mean_test_rmse
12 15 6 43460 43919 44748 44042 13 15 8 44132 44075 45010 44406 14 15 10 44374 44286
45316 44659 7 10 6 44683 44655 45657 44999 9 10 6 44683 44655 45657 44999
```

Ən yaxşı model üçün mean test RMSE balı 44,042-dir ki, bu da əvvəl default hyperparameter dəyərləri istifadə edərək əldə etdiyiniz baldan (47,019 idi) daha yaxşıdır. Təbriklər, ən yaxşı modelinizi uğurla fine-tune etdiniz!

Randomized Search

Grid search yanaşması, əvvəlki nümunədə olduğu kimi nisbətən az kombinasiya araşdırdığınız zaman yaxşıdır, lakin RandomizedSearchCV çox vaxt üstünlük verilir, xüsusilə hyperparameter axtarış məkanını (search space) böyük olduqda. Bu class GridSearchCV class-ı ilə demək olar ki, eyni şəkildə istifadə oluna bilər, lakin bütün mümkün kombinasiyaları sınamaq əvəzinə, hər iterasiyada hər hyperparameter üçün təsadüfi bir dəyər seçərək sabit sayda kombinasiyanı qiymətləndirir. Bu təəccüblü səslənə bilər, lakin bu yanaşmanın bir neçə üstünlüyü var:

- Əgər hyperparameter-lərinizdən bəziləri davamlıdırsa (və ya diskret, lakin çoxlu mümkün dəyərlə) və randomized search-i, məsələn, 1,000 iterasiya işlətməyə buraxsanız, onda o, bu hyperparameter-lərin hər biri üçün 1,000 fərqli dəyər araşdıracaq, halbuki grid search hər biri üçün sadəcə sadaladığınız bir neçə dəyəri araşdırardı.
- Tutaq ki, bir hyperparameter əslində o qədər də fərq yaratmır, lakin siz bunu hələ bilmirsiniz. Əgər onun 10 mümkün dəyəri varsa və onu grid search-ə əlavə etsəniz, onda training 10 dəfə daha uzun çəkəcək. Lakin onu random search-ə əlavə etsəniz, heç bir fərq yaratmayacaq.
- Əgər araşdırılmalı 6 hyperparameter varsa, hər biri 10 mümkün dəyərlə, onda grid search modeli bir milyon dəfə öyrətməkdən başqa çarə qoymur, halbuki random search həmişə seçdiyiniz istənilən sayda iterasiya işləyə bilər.

Hər hyperparameter üçün ya bir mümkün dəyərlər siyahısı, ya da bir ehtimal paylanması (probability distribution) təmin etməlisiniz:

```
from sklearn.model_selection import RandomizedSearchCV from scipy.stats import  
randint
```

```
param_distributions = {'preprocessing__geo__n_clusters': randint(low=3, high=50),  
'random_forest__max_features': randint(low=2, high=20)}
```

```
rnd_search = RandomizedSearchCV(  
full_pipeline, param_distributions=param_distributions, n_iter=10, cv=3,  
scoring='neg_root_mean_squared_error', random_state=42)
```

```
rnd_search.fit(housing, housing_labels)
```

Scikit-Learn-də həmçinin HalvingRandomSearchCV və HalvingGridSearchCV hyperparameter axtarış class-ları var. Onların məqsədi hesablama resurslarını daha effektiv istifadə etməkdir, ya daha sürətli öyrətmək, ya da daha böyük bir hyperparameter məkanını araşdırmaq üçün. Onlar belə işləyir: birinci raundda ya grid yanaşması, ya da random yanaşması istifadə edərək çoxlu hyperparameter kombinasiyası (“namizədlər”, candidates adlanır) yaradılır. Bu namizədlər sonra adi qaydada cross-validation istifadə

edərək qiymətləndirilən modelləri öyrətmək üçün istifadə olunur. Lakin training məhdud resurslardan istifadə edir ki, bu da bu birinci raundu əhəmiyyətli dərəcədə sürətləndirir. Default olaraq, “məhdud resurslar” modellərin training set-in kiçik bir hissəsi üzərində öyrədilməsi deməkdir. Lakin başqa məhdudiyyətlər də mümkündür, məsələn, modelin onu təyin edən bir hyperparameter-i varsa, training iterasiyalarının sayını azaltmaq. Hər namizəd qiymətləndirildikdən sonra, yalnız ən yaxşısı ikinci raunda keçir, orada rəqabət etmək üçün onlara daha çox resurs verilir. Bir neçə raunddan sonra, son namizədlər tam resurslarla qiymətləndirilir. Bu, sizə hyperparameter-ləri tənzimləyərkən bir az vaxta qənaət edə bilər.

Ensemble Metodları

Sisteminizi fine-tune etməyin başqa bir yolu, ən yaxşı işləyən modelləri birləşdirməyə çalışmaqdır. Qrup (və ya “ensemble”) çox vaxt ən yaxşı fərdi modeldən daha yaxşı işləyəcək — necə ki, random forest-lər əsaslandırıqları fərdi decision tree-lərdən daha yaxşı işləyir — xüsusilə fərdi modellər çox fərqli növ səhvlər edirsə. For example, bir k -nearest neighbors modeli öyrədib fine-tune edə, sonra sadəcə random forest proqnozunun və həmin modelin proqnozunun mean-ini proqnozlaşdıran bir ensemble modeli yarada bilərsiniz. Biz bu mövzunu Fəsil 7-də daha ətraflı əhatə edəcəyik.

Ən Yaxşı Modelləri və Onların Səhvlərini Təhlil Etmək

Ən yaxşı modelləri yoxlayaraq çox vaxt problem haqqında yaxşı içgörülər qazanacaqsınız. For example, RandomForestRegressor dəqiq proqnozlar vermək üçün hər attribute-un nisbi əhəmiyyətini göstərə bilər:

```
>>> final_model = rnd_search.best_estimator # includes preprocessing_ >>>
feature_importances = final_model["random_forest"].feature_importances_ >>>
feature_importances.round(2) array([0.07, 0.05, 0.05, 0.01, 0.01, 0.01, 0.01, 0.19, [...], 0.01])
```

Gəlin bu əhəmiyyət ballarını azalan ardıcılıqla sıralayaq və onları müvafiq attribute adlarının yanında göstərək:

```
>>> sorted(zip(feature_importances, ...
final_model["preprocessing"].get_feature_names_out()), ... reverse= True ) ...
[(0.18694559869103852, 'log__median_income'), (0.0748194905715524,
'cat__ocean_proximity_INLAND'), (0.06926417748515576, 'bedrooms__ratio'),
(0.05446998753775219, 'rooms_per_house__ratio'), (0.05262301809680712,
'people_per_house__ratio'), (0.03819415873915732, 'geo__Cluster 0 similarity'), [...],
(0.00015061247730531558, 'cat__ocean_proximity_NEAR BAY'),
(7.301686597099842e-05, 'cat__ocean_proximity_ISLAND')]
```

Bu məlumatla, daha az faydalı feature-lardan bəzilərini silməyi sınaq etmək istəyə bilərsiniz (məsələn, görünür yalnız bir ocean_proximity kateqoriyası həqiqətən faydalıdır, ona görə

də digərlərini silməyi sınaya bilərsiniz).

MƏSLƏHƏT

`sklearn.feature_selection.SelectFromModel` transformer-i sizin üçün ən az faydalı feature-ları avtomatik silə bilər: onu fit etdiyiniz zaman, o, bir model (adətən bir random forest) öyrədir, onun `feature_importances_` attribute-una baxır və ən faydalı feature-ları seçir. Sonra `transform()` çağırduğunuz zaman digər feature-ları silir.

Sisteminizin etdiyi konkret səhvlərə də baxmalısınız, sonra onları niyə etdiyini və problemi nəyin həll edə biləcəyini anlamağa çalışmalısınız: əlavə feature-lar əlavə etmək və ya məlumsız (uninformative) olanlardan qurtulmaq, outlier-ləri təmizləmək və s.

İndi həm də modelinizin yalnız orta hesabla deyil, həm də bütün rayon kateqoriyalarında — istər kənd, istər şəhər, istər varlı, istər kasıb, istər şimal, istər cənub, istər azlıq, istərsə də deyil — yaxşı işlədiyini təmin etmək üçün yaxşı vaxtdır. Validation set-inizin hər kateqoriya üçün altçoxluqlarını yaratmaq bir az iş tələb edir, lakin vacibdir: əgər modeliniz rayonların bütöv bir kateqoriyasında zəif işləyirsə, onda problem həll olunana qədər çox güman ki, deploy olunmamalıdır, və ya heç olmasa həmin kateqoriya üçün proqnozlar vermək üçün istifadə olunmamalıdır, çünki faydadan çox zərər verə bilər.

Sisteminizi Test Set Üzərində Qiymətləndir

Modellərinizi bir müddət tənziqlədikdən sonra, nəhayət kifayət qədər yaxşı işləyən bir sisteminiz olur. İndi son modeli test set üzərində qiymətləndirməyə hazırsınız. Bu prosesdə xüsusi heç nə yoxdur; sadəcə test set-inizdən predictor-ları və label-ları əldə edin və datanı transformasiya edib proqnozlar vermək üçün `final_model`-inizi işlədin, sonra bu proqnozları qiymətləndirin:

```
X_test = strat_test_set.drop("median_house_value", axis=1) y_test = strat_test_set["median_house_value"].copy()
```

```
final_predictions = final_model.predict(X_test)
```

```
final_rmse = mean_squared_error(y_test, final_predictions, squared=False )  
print(final_rmse) # prints 41424.40026462184
```

Bəzi hallarda, generalization error-un belə bir nöqtə qiymətləndirməsi (point estimate) sizi işə salmağa inandırmaq üçün tam kifayət etməyəcək: əgər o, hazırda production-da olan modeldən sadəcə 0.1% daha yaxşıdırsa nə olar? Bu qiymətləndirmənin nə qədər dəqiq olduğu barədə təsəvvürə sahib olmaq istəyə bilərsiniz. Bunun üçün `scipy.stats.t.interval()` istifadə edərək generalization error üçün 95% *confidence interval* hesablaya bilərsiniz. 39,275-dən 43,467-ə qədər kifayət qədər böyük bir interval alırsınız və əvvəlki nöqtə qiymətləndirməniz olan 41,424 təxminən onun ortasındadır:

```
>>> from scipy import stats >>> confidence = 0.95 >>> squared_errors = (final_predictions - y_test) ** 2 >>> np.sqrt(stats.t.interval(confidence, len(squared_errors) - 1, ...  
loc=squared_errors.mean(), ... scale=stats.sem(squared_errors))) ... array([39275.40861216,  
43467.27680583])
```

Əgər çoxlu hyperparameter tuning etmişsinizsə, performans adətən cross-validation istifadə edərək ölçdüyünüzdən bir az daha pis olacaq. Bunun səbəbi, sisteminizin validation datasında yaxşı işləmək üçün fine-tune olunması və çox güman ki, naməlum dataset-lərdə o qədər də yaxşı işləməyəcəyidir. Bu nümunədə belə deyil, çünki test RMSE validation RMSE-dən aşağıdır, lakin bu baş verdikdə, rəqəmləri test set-də yaxşı göstərmək üçün hyperparameter-ləri tənzimləmək vəsvəsəsinə müqavimət göstərməlisiniz; təkmilləşmələrin yeni dataya ümumiləşmə (generalize) ehtimalı azdır.

İndi layihənin işə salmadan əvvəlki (prelaunch) fazası gəlir: həllinizi təqdim etməli (öyrəndiklərinizi, nəyin işlədiyini və nəyin işləmədiyini, hansı fərziyyələrin edildiyini və sisteminizin məhdudiyyətlərinin nə olduğunu vurğulayaraq), hər şeyi sənədləşdirməli və aydın vizuallaşdırmalar və yadda saxlanması asan ifadələrlə (məsələn, “median income ev qiymətlərinin bir nömrəli predictor-udur”) gözəl təqdimatlar yaratmalısınız. Bu California housing nümunəsində, sistemin son performansını çox vaxt 30% yanlış olan ekspertlərin qiymət təxminlərindən çox da yaxşı deyil, lakin onu işə salmaq yenə də yaxşı ideya ola bilər, xüsusilə bu, ekspertlərə daha maraqlı və məhsuldar tapşırıqlar üzərində işləmək üçün bir az vaxt boşaltsa.

Sisteminizi İşə Sal, Nəzarət Et və Saxlanılmasını Təmin Et

Mükəmməl, işə salmaq üçün təsdiq aldınız! İndi həllinizi production üçün hazır etməlisiniz (məsələn, kodu cilalamaq, sənədlər və test-lər yazmaq və s.). Sonra modelinizi production mühitinizə deploy edə bilərsiniz. Bunu etməyin ən sadə yolu, öyrətdiyiniz ən yaxşı modeli saxlamaq, faylı production mühitinizə köçürmək və onu yükləməkdir. Modeli saxlamaq üçün joblib kitabxanasını belə istifadə edə bilərsiniz:

```
import joblib
```

```
joblib.dump(final_model, "my_california_housing_model.pkl")
```

MƏSLƏHƏT

Təcrübə apardığınız hər modeli saxlamaq çox vaxt yaxşı ideyadır ki, istədiyiniz istənilən modelə asanlıqla geri qayıda bilərsiniz. Cross-validation ballarını və ola bilsin validation set üzərində faktiki proqnozları da saxlaya bilərsiniz. Bu, sizə model növləri arasında balları asanlıqla müqayisə etməyə və etdikləri səhv növlərini müqayisə etməyə imkan verəcək.

Modeliniz production-a köçürüldükdən sonra, onu yükləyib istifadə edə bilərsiniz. Bunun üçün əvvəlcə modelin əsaslandığı istənilən custom class və funksiyaları import etməlisiniz

(yəni kodu production-a köçürməlisiniz), sonra modeli joblib istifadə edərək yükləməli və onu proqnozlar vermək üçün istifadə etməlisiniz:

import joblib

[...] *# import KMeans, BaseEstimator, TransformerMixin, rbf_kernel, etc.*

def column_ratio(X): [...] **def ratio_name(function_transformer, feature_names_in):** [...]

class ClusterSimilarity (BaseEstimator, TransformerMixin): [...]

final_model_reloaded = joblib.load("my_california_housing_model.pkl")

new_data = [...] *# some new districts to make predictions for* predictions =
final_model_reloaded.predict(new_data)

For example, bəlkə model bir veb saytın içində istifadə olunacaq: istifadəçi yeni bir rayon haqqında bəzi data yazacaq və Estimate Price düyməsinə klik edəcək. Bu, datanı ehtiva edən bir sorğunu veb serverə göndərəcək, o da onu veb tətbiqinizə ötürəcək və nəhayət kodunuz sadəcə modelin predict() metodunu çağıracaq (modeli hər dəfə istifadə olunduqda deyil, server başladıqda yükləmək istəyirsiniz). Alternativ olaraq, modeli veb tətbiqinizin bir REST API¹³ vasitəsilə sorğulaya biləcəyi xüsusi bir veb xidmətinə (web service) büküb istifadə edə bilərsiniz (Şəkil 2-20-yə bax). Bu, əsas tətbiqi kəsmədən modelinizi yeni versiyalara yeniləməyi asanlaşdırır. O, həmçinin miqyaslandırmanı (scaling) sadələşdirir, çünki lazım olduqca istənilən qədər veb xidməti başlada və veb tətbiqinizdən gələn sorğuları bu veb xidmətləri arasında yük balanslaşdırma (load-balance) bilirsiniz. Üstəlik, o, veb tətbiqinizə yalnız Python deyil, istənilən proqramlaşdırma dilindən istifadə etməyə imkan verir.

Şəkil 2-20. Veb xidməti kimi deploy olunmuş və bir veb tətbiqi tərəfindən istifadə olunan model

Başqa bir populyar strategiya, modelinizi buluda (cloud), məsələn Google-un Vertex AI-na (əvvəllər Google Cloud AI Platform və Google Cloud ML Engine kimi tanınırdı) deploy etməkdir: sadəcə modelinizi joblib istifadə edərək saxlayın və Google Cloud Storage-ə (GCS) yükləyin, sonra Vertex AI-a keçin və GCS faylına işarə edən yeni bir model versiyası yaradın. Vəssalam! Bu, sizə yük balanslaşdırma və miqyaslandırma ilə məşğul olan sadə bir veb xidməti verir. O, input datasını (məsələn, bir rayonun) ehtiva edən JSON sorğularını qəbul edir və proqnozları ehtiva edən JSON cavabları qaytarır. Sonra bu veb xidmətini veb saytıınızda (və ya istifadə etdiyiniz hər hansı production mühitində) istifadə edə bilərsiniz. Fəsil 19-da görəcəyiniz kimi, TensorFlow modellərini Vertex AI-da deploy etmək Scikit-Learn modellərini deploy etməkdən çox da fərqli deyil.

Lakin deployment hekayənin sonu deyil. Sisteminizin canlı (live) performansını müntəzəm intervallarla yoxlamaq və düşdükdə xəbərdarlıqlar (alerts) işə salmaq üçün monitoring kodu da yazmalısınız. O, çox sürətlə düşə bilər, məsələn infrastrukturunuzda bir komponent sınırsa, lakin nəzərə alın ki, o, həm də çox yavaş çürüyə (decay) bilər, ki, bu da

uzun müddət asanlıqla fərq edilməyə bilər. Bu, model rot (model çürüməsi) səbəbindən kifayət qədər yaygındır: əgər model keçən ilin datası ilə öyrədilibsə, o, bugünkü dataya uyğunlaşmaya bilər.

Beləliklə, modelinizin canlı performansını monitorinq etməlisiniz. Bəs bunu necə edirsiniz? Yaxşı, asılıdır. Bəzi hallarda, modelin performansı downstream metriklərdən çıxarıla (inferred) bilər. For example, əgər modeliniz bir tövsiyə sisteminin (recommender system) bir hissəsidirsə və istifadəçilərin maraqlana biləcəyi məhsullar təklif edirsə, onda hər gün satılan tövsiyə olunmuş məhsulların sayını monitorinq etmək asandır. Əgər bu rəqəm düşürsə (tövsiyə olunmamış məhsullarla müqayisədə), onda əsas şübhəli modeldir. Bu, data pipeline-ın sınması, və ya bəlkə modelin təzə data üzərində yenidən öyrədilməsi (retrained) ehtiyacı (qısa zamanda müzakirə edəcəyimiz kimi) səbəbindən ola bilər.

Lakin modelin performansını qiymətləndirmək üçün insan təhlilinə (human analysis) də ehtiyacınız ola bilər. For example, tutaq ki, bir istehsal xəttində müxtəlif məhsul qüsurlarını aşkar etmək üçün bir şəkil təsnifatı (image classification) modeli (Fəsil 3-də bunlara baxacağıq) öyrətdiniz. Minlərlə qüsurlu məhsul müştərilərinizə göndərilməzdən əvvəl, modelin performansı düşdükdə necə xəbərdarlıq ala bilərsiniz? Bir həll, modelin təsnif etdiyi bütün şəkillərin bir nümunəsini (xüsusilə modelin o qədər də əmin olmadığı şəkilləri) insan qiymətləndiricilərə (human raters) göndərməkdir. Tapşırıqdan asılı olaraq, qiymətləndiricilər ekspertlər olmalı, və ya qeyri-mütəxəssislər ola bilərlər, məsələn bir crowdsourcing platformasındakı işçilər (məsələn, Amazon Mechanical Turk). Bəzi tətbiqlərdə onlar hətta istifadəçilərin özləri ola bilər, məsələn sorğular və ya yenidən təyinatlı captcha-lar vasitəsilə cavab verərək.¹⁴

Hər halda, bir monitoring sistemi qurmalısınız (canlı modeli qiymətləndirmək üçün insan qiymətləndiriciləri ilə və ya onlarsız), həmçinin uğursuzluqlar halında nə edəcəyini və onlara necə hazırlaşacağını müəyyən etmək üçün bütün müvafiq prosesləri. Təəssüf ki, bu çoxlu iş ola bilər. Əslində, bu çox vaxt bir modeli qurmaq və öyrətməkdən çox daha çox işdir.

Əgər data təkamül etməyə (evolving) davam edirsə, dataset-lərinizi yeniləməli və modelinizi müntəzəm olaraq yenidən öyrətməli olacaqsınız. Çox güman ki, bütün prosesi mümkün qədər avtomatlaşdırmalısınız. Avtomatlaşdırma biləcəyiniz bir neçə şey:

Təzə datanı müntəzəm olaraq toplayın və ona label verin (məsələn, insan qiymətləndiriciləri istifadə edərək).

- Modeli öyrətmək və hyperparameter-ləri avtomatik fine-tune etmək üçün bir skript yazın. Bu skript ehtiyaclarınızdan asılı olaraq, məsələn hər gün və ya hər həftə avtomatik işləyə bilər.
- Yenilənmiş test set üzərində həm yeni modeli, həm də əvvəlki modeli qiymətləndirəcək, və performans azalmadıqda (azalıbsa, niyə olduğunu araşdırdığınızdan əmin olun) modeli production-a deploy edəcək başqa bir skript yazın. Skript çox güman ki, modelinizin performansını test set-in müxtəlif

altçoxluqları üzərində test etməlidir, məsələn kasıb və ya varlı rayonlar, kənd və ya şəhər rayonları və s.

Modelin input data keyfiyyətini də qiymətləndirdiyinizdən əmin olmalısınız. Bəzən performans zəif keyfiyyətli bir siqnal səbəbindən bir az pisləşəcək (məsələn, təsadüfi dəyərlər göndərən nasaz bir sensor, və ya başqa bir komandanın çıxışının köhnəlməsi), lakin sisteminizin performansının bir xəbərdarlıq işə salacaq qədər pisləşməsi bir müddət çəke bilər. Modelinizin input-larını monitorinq etsəniz, bunu daha erkən tuta bilərsiniz. For example, getdikcə daha çox input-da bir feature çatışmırsa, və ya mean və ya standard deviation training set-dən çox uzaqlaşarsa, və ya bir categorical feature yeni kateqoriyalar ehtiva etməyə başlayırsa bir xəbərdarlıq işə sala bilərsiniz.

Nəhayət, yaratdığınız hər modelin yedəklərini (backups) saxladığınızdan və yeni model hansısa səbəbdən pis şəkildə uğursuz olmağa başlayarsa, tez bir zamanda əvvəlki modelə geri qayıtmaq (roll back) üçün prosesə və alətlərə sahib olduğunuzdan əmin olun. Yedəklərə sahib olmaq həm də yeni modelləri əvvəlkilərlə asanlıqla müqayisə etməyi mümkün edir. Eyni şəkildə, dataset-lərinizin hər versiyasının yedəklərini saxlamalısınız ki, yeni dataset hansısa anda korlanırsa (məsələn, ona əlavə olunan təzə datanın outlier-lərlə dolu olduğu ortaya çıxarsa) əvvəlki dataset-ə geri qayıda bilərsiniz. Dataset-lərinizin yedəklərinə sahib olmaq həmçinin istənilən model istənilən əvvəlki dataset-ə qarşı qiymətləndirməyə imkan verir.

Gördüyünüz kimi, machine learning kifayət qədər çox infrastruktur tələb edir. Fəsil 19 bunun bəzi aspektlərini müzakirə edir, lakin bu, *ML Operations* (MLOps) adlanan çox geniş bir mövzudur və öz kitabına layiqdir. Ona görə də ilk ML layihənin qurulması və production-a deploy edilməsi çoxlu səy və vaxt aparırsa təəccüblənməyin. Xoşbəxtlikdən, bütün infrastruktur qurulduqdan sonra, ideyadan production-a getmək çox daha sürətli olacaq.

Sınyayın! (Try It Out!)

Ümid edirəm bu fəsil sizə bir machine learning layihəsinin necə göründüyü barədə yaxşı təsəvvür verdi, həmçinin əla bir sistem öyrətmək üçün istifadə edə biləcəyiniz bəzi alətləri göstərdi. Gördüyünüz kimi, işin çoxu data hazırlığı (data preparation) addımındadır: monitoring alətləri qurmaq, insan qiymətləndirmə pipeline-ları yaratmaq və müntəzəm model training-i avtomatlaşdırmaq. Machine learning alqoritmləri əlbəttə vacibdir, lakin çox güman ki, ümumi proseslə rahat olmaq və üç-dörd alqoritmə yaxşı bilmək, bütün vaxtınızı qabaqcıl alqoritmləri araşdırmağa sərf etməkdən daha üstündür.

Beləliklə, əgər artıq etməmişsinizsə, indi bir laptop götürmək, maraqlandığınız bir dataset seçmək və bütün prosesi A-dan Z-yə qədər keçməyə cəhd etmək üçün yaxşı vaxtdır. Başlamaq üçün yaxşı bir yer Kaggle kimi bir yarışma (competition) veb saytıdır: oynamaq üçün bir dataset-iniz, aydın bir məqsədiniz və təcrübəni bölüşmək üçün insanların olacaq. Əylənin!

Tapşırıqlar (Exercises)

Aşağıdakı tapşırıqlar bu fəslin housing dataset-inə əsaslanır:

1. Müxtəlif hyperparameter-lərlə bir support vector machine regressor (`sklearn.svm.SVR`) sınaqın, məsələn `kernel="linear"` (C hyperparameter-i üçün müxtəlif dəyərlərlə) və ya `kernel="rbf"` (C və gamma hyperparameter-ləri üçün müxtəlif dəyərlərlə). Qeyd edin ki, support vector machine-lər böyük dataset-lərə yaxşı miqyaslanmır, ona görə də çox güman ki, modelinizi yalnız training set-in ilk 5,000 instance-ı üzərində öyrətməli və yalnız 3-fold cross-validation istifadə etməlisiniz, əks halda saatlarla çəkəcək. Hyperparameter-lərin nə demək olduğu barədə hələlik narahat olmayın; onları Fəsil 5-də müzakirə edəcəyik. Ən yaxşı SVR predictor necə işləyir?
2. `GridSearchCV`-ni `RandomizedSearchCV` ilə əvəz etməyi sınaqın.
3. Yalnız ən vacib attribute-ları seçmək üçün preparation pipeline-ına bir `SelectFromModel` transformer əlavə etməyi sınaqın.
4. `fit()` metodunda bir *k*-nearest neighbors regressor (`sklearn.neighbors.KNeighborsRegressor`) öyrədən və `transform()` metodunda modelin proqnozlarını verən bir custom transformer yaratmağı sınaqın. Sonra bu transformer-ə input kimi latitude və longitude istifadə edərək bu feature-u preprocessing pipeline-ına əlavə edin. Bu, modelə ən yaxın rayonların median ev qiymətinə uyğun gələn bir feature əlavə edəcək.
5. `GridSearchCV` istifadə edərək bəzi preparation seçimlərini avtomatik araşdırın.
6. `StandardScalerClone` class-ını yenidən sıfırdan implement etməyə çalışın, sonra `inverse_transform()` metodu üçün dəstək əlavə edin:
`scaler.inverse_transform(scaler.fit_transform(X))` işlətmək *X*-ə çox yaxın bir array qaytarmalıdır. Sonra feature adları üçün dəstək əlavə edin: input bir `DataFrame`-dirsə, `fit()` metodunda `feature_names_in_` təyin edin. Bu attribute sütun adlarının bir `NumPy` array-i olmalıdır. Nəhayət, `get_feature_names_out()` metodunu implement edin: onun bir optional input `features=None` argumenti olmalıdır. Ötürülərsə, metod onun uzunluğunun `n_features_in_` ilə uyğun olduğunu yoxlamalı və təyin olunubsa `feature_names_in_` ilə uyğun olmalıdır; sonra input `features` qaytarılmalıdır. Əgər input `features` `None`-dursə, onda metod ya təyin olunubsa `feature_names_in_` qaytarmalı, ya da əks halda `n_features_in_` uzunluğunda `np.array(["x0", "x1", ...])` qaytarmalıdır.

Bu tapşırıqların həlləri bu fəslin notebook-unun sonunda, <https://homl.info/colab3> ünvanında mövcuddur.

- 1 Orijinal dataset R. Kelley Pace və Ronald Barry, "Sparse Spatial Autoregressions", *Statistics & Probability Letters* 33, no. 3 (1997): 291–297 məqaləsində dərc olunub.

- 2 Bir machine learning sisteminə verilən bir parça məlumat çox vaxt *signal* adlanır, Claude Shannon-un Bell Labs-da telekommunikasiyaları yaxşılaşdırmaq üçün hazırladığı *information theory*-yə istinadən. Onun nəzəriyyəsi: yüksək signal-to-noise nisbəti istəyirsiniz.
- 3 Xatırlayın ki, transpose operatoru bir sütun vektorunu bir sətir vektoruna çevirir (və əksinə).
- 4 Hüquqi məhdudiyyətləri də yoxlamalı ola bilərsiniz, məsələn heç vaxt təhlükəsiz olmayan data store-lara kopyalanmamalı olan şəxsi (private) sahələr.
- 5 Standard deviation ümumiyyətlə σ (yunan hərfi siqma) ilə işarələnir və o, *variance*-in kvadrat köküdür, ki, *variance* da mean-dən kvadratik sapmanın (deviation) ortalamasıdır. Bir feature zəng formalı *normal distribution*-a (həmçinin *Gaussian distribution* adlanır) malik olduqda, ki bu çox yaygındır, “68-95-99.7” qaydası tətbiq olunur: dəyərlərin təxminən 68%-i mean-in 1σ daxilində, 95%-i 2σ daxilində, 99.7%-i isə 3σ daxilində qalır.
- 6 Çox vaxt insanların random seed-i 42-yə təyin etdiyini görəcəksiniz. Bu rəqəmin Həyatın, Kainatın və Hər Şeyin Əsas Sualına Cavab olmaqdan başqa heç bir xüsusi xassəsi yoxdur.
- 7 Yerləşmə məlumatı əslində kifayət qədər kobuddur (coarse), və nəticədə bir çox rayonun tam eyni ID-si olacaq, ona görə də onlar eyni set-də (test və ya train) bitəcəklər. Bu, bir az təəssüfləndirici sampling bias daxil edir.
- 8 Əgər bunu boz tonlarda (grayscale) oxuyursunuzsa, qırmızı bir qələm götürün və Bay Area-dan San Diego-ya qədər sahil xəttinin çoxunu cızın (gözlədiyiniz kimi). Sacramento ətrafına bir az sarı ləkə də əlavə edə bilərsiniz.
- 9 Dizayn prinsipləri haqqında daha çox detal üçün Lars Buitinck et al., “API Design for Machine Learning Software: Experiences from the Scikit-Learn Project”, arXiv preprint arXiv:1309.0238 (2013) məqaləsinə baxın.
- 10 Bəzi predictor-lar həm də proqnozlarının etibarlılığını (confidence) ölçmək üçün metodlar təqdim edir.
- 11 Bu sətirləri oxuyana qədər, bütün transformer-lərin input kimi bir DataFrame aldıqda Pandas DataFrame-lər vermələrini mümkün etmək mümkün ola bilər: Pandas in, Pandas out. Bunun üçün çox güman ki, qlobal bir konfigurasiya seçimi olacaq: `sklearn.set_config(pandas_in_out=True)`.
- 12 Daha çox detal üçün SciPy sənədlərinə baxın.
- 13 Qısaca, bir REST (və ya RESTful) API, bəzi konvensiyalara tabe olan HTTP-əsaslı bir API-dir, məsələn resursları oxumaq, yeniləmək, yaratmaq və ya silmək üçün standart HTTP verb-lərindən (GET, POST, PUT və DELETE) istifadə etmək və input-lar və output-lar üçün JSON istifadə etmək.
- 14 Captcha bir istifadəçinin robot olmadığından əmin olmaq üçün bir testdir. Bu testlər çox vaxt training datasına label vermənin ucuz bir yolu kimi istifadə olunub.